一、国产商用密码与抗量子密码混合的 TLS 协议源码库

1. 《国产商用密码与抗量子密码混合的 TLS 协议源码库设计报告》
2. 《国产商用密码与抗量子密码混合的 TLS 协议源码库测试大纲》

二、国产商用密码与抗量子密码混合的 IPSec 协议源码库

1. 《国产商用密码与抗量子密码混合的 IPSec 协议源码库设计报告》
2. 《国产商用密码与抗量子密码混合的 IPSec 协议源码库测试大纲》

三、国产商用密码与抗量子密码混合的 SSH 协议源码库

1. 《国产商用密码与抗量子密码混合的 SSH 协议源码库设计报告》
2. 《国产商用密码与抗量子密码混合的 SSH 协议源码库测试大纲》

国产商用密码与抗量子密码混合的TLS协议源码库设计报告

编　　写 _____

校　　对 _____

批　　准 _____

文档控制

变更记录

版本号	创建/修改日期	版本控制人	增加/修改/删除	简单描述
V1.0	2025.04.09	刘凯博	增加	设计报告初稿
V1.1	2026.06.19	刘凯博	增加	调研工作完善
V1.2	2026.07.05	刘凯博	修改	方案设计完善
V1.3	2026.07.10	张航	修改	报告格式调整

目录

1. 设计目标 1
2. 背景现状 2
2.1. 经典认证密钥交换协议研究现状 3
2.1.1. 经典认证方式 3
2.1.2. 经典密钥交换协议 7
2.1.3. 小结 8
2.2. 抗量子认证密钥交换协议研究现状 8
2.2.1. 抗量子认证方式 9
2.2.2. 小结 12
2.3. 混合抗量子认证密钥交换协议研究现状 12
2.3.1. 混合认证方式 13
2.3.2. 混合密钥交换方式 14
2.4. TLS 协议抗量子密码计算密码迁移研究现状 18
2.5. TLS 协议抗量子迁移存在的问题及挑战 22
3. 抗量子 TLS 协议方案设计 23
3.1. 密码协议基本信息调研 24
3.2. 迁移点分析与确认 27
3.3. 方案设计 27
3.3.1. 基于签名的抗量子 TLS 协议设计方案 28
3.3.2. 流程优化的抗量子 TLS 协议设计方案 30
3.3.3. 异构认证的抗量子 TLS 协议设计方案 36
4. 分层模块详细功能设计 40
4.1. 公共基础工具支撑层 41
4.2. 混合密码算法支撑层 41
4.3. TLS 握手核心改造层 41
4.4. TLS 调度交互层 42
4.5. 模块整体交互执行层 42
5. TLS 协议抗量子迁移实现 43
5.1. ClientHello 和 ServerHello 的字段解析 43
5.2. 协议扩展与标识定义 45
5.3. ClientHello 和 ServerHello 消息改造 45
5.3.1. ClientHello 密钥交换相关改造 45
5.3.2. ClientHello 签名协商字段改造 45
5.3.3. ServerHello 密钥处理改造 46
5.3.4. ServerHello 签名协商处理 46
5.4. 预主密钥生成流程 46
5.4.1. 密钥派生计算 46
5.4.2. 混合签名校验流程（CertificateVerify） 46
5.5. 兼容性与安全性设计 47
5.5.1. 协商回退机制 47
5.5.2. 安全性增强 47
5.6. 协议改造影响分析 47

5.6.1. 消息长度分析 47
5.6.2. 网络性能分析 47
5.6.3. 兼容性分析 48
5.6.4. 安全性分析 48
5.7. 协议模块实现方案 48
6. 参考文献 63

图目录

图 1（非）分离混合签名方案 13
图 2 完整的 TLS 1.2 握手的消息流程 15
图 3 握手过程中的额外数据交换 15
图 4 混合 TLS 协议中的信息流 18
图 5 混合 TLS1.3 流程示意图 29
图 6 流程优化的抗量子 TLS 协议设计方案流程图 31
图 7 异构认证模式 TLS 协议（Sig-KEM） 37
图 8 异构认证模式 TLS 协议（KEM-Sig） 38
图 9 抗量子 TLS 协议整体框架 40

1. 设计目标

TLS 协议是当前互联网应用部署范围最广、用户规模最大的安全传输基础协议。TLS 协议承载着全球 90% 以上通信流量的安全保障任务，全面支撑网页访问、金融支付、物联网、移动终端、政企内网等多场景安全通信。面向抗量子迁移改造时，TLS 体系改造需同步完成密钥交换算法迭代、协议扩展改造、上下游生态适配、存量系统平滑过渡等多重复杂工程工作，其改造复杂度、行业影响广度远高于单一密码算法替换，是抗量子密码规模化落地中极具代表性与挑战性的核心场景。

然而，在抗量子 TLS 协议迁移的工程实践中，现有工作多集中于混合密钥交换算法设计替换，缺乏对于不同认证方式、不同迁移模式（协议交互流程）下稳健性、平滑性的系统性探索。具体而言，如何解决抗量子算法尺寸过大导致的 TLS 握手中断问题，如何实现异构认证模式（如混合签名与密钥封装）下的平滑互通，现有研究仍存在技术短板。基于此，本课题针对协议平滑迁移中的技术难题开展针对性研究：

一方面，以存量业务稳定运行作为前置约束，充分兼容传统 RSA、ECC 及国产商用密码 SM2 等经典密码算法，不中断现有网银、手机银行等线上金融业务；同时全面匹配量子计算攻击下的安全防护要求，构建国产商用密码与抗量子密码融合的混合协议架构，形成适配银行业场景、安全可控的 TLS 抗量子混合密码协议，兼顾国产化合规要求与抗量子安全防护能力。

另一方面，分别开展协议交互流程优化、异构认证兼容适配两类机制研究，针对性化解抗量子算法数据膨胀引发的握手中断、节点改造进度失衡造成的异构节点互通困难两大核心问题。

最终，整合混合协议架构、抗分包握手优化、异构节点兼容适配等全部理论与机制研究成果完成工程化集成落地，构建了一套完整的抗量子 TLS 迁移解决方案，并同步研发交付国产商用密码与抗量子密码混合的 TLS 协议源码库，该源码库能够支持 2 类抗量子密码算法，覆盖 128、256 比特的量子安全等级。整套方案在保障抗量子安全的前提下，兼顾兼容性与部署灵活性，为金融行业 TLS 协议后量子平滑迁移提供完备理论支撑与可落地工程实施路径。

2. 背景现状

密码技术作为信息安全的基石，在金融领域中得到广泛应用，主要用于保护关键业务数据在传输和存储过程中的真实性、机密性、完整性和不可否认性。

但是，随着量子技术的快速发展，理论上能够在短时间内解决传统计算机难以处理的复杂数学问题，如大数分解和离散对数问题。Shor 算法[1]已经证明量子计算机可在多项式时间内求解整数分解和离散对数问题，这影响到当前公钥密码体系（如 RSA[2]、ECC[3]等）安全性的基础，使 RSA 和 ECC 等传统公钥密码机制面临被破解的风险。

基于以上威胁，全球主要的国家已经开始采用“先收集后破译”的策略，即利用现有的技术手段尽可能多地收集加密通信数据，等待量子技术成熟后再进行解密分析。这种策略加剧了金融领域的紧迫性，因为一旦量子技术突破瓶颈，大量历史数据和当前敏感信息都将面临暴露的风险，严重威胁国家安全、社会稳定和个人隐私，金融领域信息系统的前向安全性受到了严峻挑战。

目前金融领域中常用的密码协议有 TLS[4]、IPSec[5]、SSH[6]、SFTP 等，其中，TLS 协议应用比较广泛。TLS 协议主要对传输中的金融数据进行机密性保护和完整性保护，即使数据在传输过程中被截获，也无法被未授权方轻易解密或篡改，从而有效保护数据的机密性和完整性。该协议通过数字证书的身份验证机制，确保用户访问的是真实、合法的金融机构网站，而非钓鱼网站或恶意软件的伪装。这降低了用户遭受欺诈的风险，增强了用户对金融机构的信任度。此外，TLS 协议利用数字签名技术对传输的数据进行完整性校验，确保数据在传输过程中未被篡改或破坏。这保障了数据的真实性和可靠性，对于金融交易等需要确保数据准确性的场景尤为重要。

然而，随着量子计算技术从理论加速迈向工程化突破，现行 TLS 协议所依赖的经典密码体系正面临前所未有的生存危机。以 Shor 算法为核心的量子攻击手段，能够在多项式时间内破解 RSA、ECC 等公钥算法，TLS 协议使用的签名、数字证书等技术，均基于传统的公钥密码体制，无法抵抗量子攻击，使用 TLS 协

议传输的大量敏感信息也无法抵御“先收集后破译”的攻击，而全球超 90%的 HTTPS 流量仍基于 TLS1.3 协议。

为应对以上需求，本报告主要研究密码协议的抗量子迁移工作及实践。具体地，本报告设计密码协议抗量子迁移实践策略和流程，并针对 TLS 协议进行具体的抗量子迁移设计、实现和测试，实现 128bit 安全强度的密码协议抗量子迁移。

2.1. 经典认证密钥交换协议

1976 年，Diffie 和 Hellman 首次提出了密钥交换的概念，并提出了著名的 Diffie-Hellman 协议[7]，是第一个实用的公开密钥交换协议，允许没有共享秘密的通信方在不安全的信道上生成一个共享的秘密密钥。基本的 Diffie-Hellman 协议不提供通信双方的身份认证，容易受到中间人等攻击。为了防止此类攻击，认证密钥交换协议加入了对通信双方身份真实性验证的过程，可以让参与方在建立共享通信密钥的同时，确认对方的身份真实性。

Needham 和 Schroeder 于 1978 年提出了 Needham-Schroeder 协议[8]，这是一种基于对称加密的密钥交换协议，分为对称密钥版本（NS-SK）和公开密钥版本（NS-PK），通过第三方认证服务器实现了安全的密钥交换[9]。20 世纪 80 年代，麻省理工学院（MIT）开发了 Kerberos 协议，这是一种基于对称密钥的分布式密钥交换协议，通过一个可信的第三方（称为密钥分发中心，KDC）提供身份验证服务，广泛应用于企业网络环境。在 1992 年，Diffie、Van Oorschot 和 Wiener 提出了 Station-to-Station (STS)协议[10]，该协议结合数字签名和 Diffie-Hellman 密钥交换，实现了双向身份验证，并防止中间人攻击。在 1993 年，IETF（Internet Engineering Task Force）开发了 IKE（Internet Key Exchange）协议[11]，该协议结合了 ISAKMP（Internet Security Association and Key Management Protocol）和 Oakley 协议[12]，提供了身份验证和密钥交换功能，并用于在 IPsec 中建立安全关联。这些协议在不同的应用场景中发挥着重要作用，为现代密码学和网络安全奠定了坚实的基础。

2.1.1. 经典认证方式

身份认证是确保系统安全和用户身份的关键技术，该技术通过不同的机制验证用户的身份信息，从而防止未经授权的访问。经典的认证方式主要包括基于签名算法的认证、基于口令的认证、基于令牌的认证、基于生物特征的认证和多种认证方式相结合的多因子认证。

基于公钥密码算法的认证方式最早由 Whitfield Diffie 和 Martin Hellman 于 1976 年提出，该方式通过公钥和私钥的签名验签过程来验证用户身份，为公钥密码算法的应用奠定了基础[13]。在此基础上又有了基于签名、口令、令牌、生物特征的认证方式。

基于口令的认证方式是最早和最广泛使用的认证方法之一，用户通过输入预先设定的口令来验证身份。这个概念可以追溯到计算机系统的早期发展阶段，在 20 世纪 60 年代和 70 年代，口令已经被广泛用于计算机系统中[14]。

基于令牌的认证方式通过分发和验证令牌来确认用户身份。OAuth 协议是这种方式的一个典型应用，它最早由 Blaine Cook 等人于 2010 年提出，用于授权第三方应用访问用户资源而无需暴露用户的凭证[15]。

基于生物特征的认证方式利用个体独特的生理或行为特征进行身份验证。John Daugman 于 1993 年提出了基于虹膜识别的生物特征认证技术，这项技术被广泛认为是生物特征认证领域的重要突破之一[16]。

基于多因子认证（MFA）结合了两种或多种不同类型的验证因素来确认用户身份，显著提高了安全性。MFA 的概念虽然没有一个明确的单一提出者，但其应用和推广可以追溯到 20 世纪 90 年代末期和 21 世纪初期，特别是在企业网络安全和金融服务领域[17]。

而在我国，提出的 GB/T15843 标准中，主要涉及信息技术和安全技术中的实体鉴别机制。这个标准分为多个部分，每部分涵盖不同的鉴别机制。其中对于认证方式做出了一些定义其中包括采用对称加密算法、数字签名技术、密码校验函数和零知识技术的实体鉴别机制。

1、对称加密算法：这种方法使用相同的密钥进行加密和解密。对称加密算法可以用于单向和双向鉴别。在单向鉴别中，客户端使用共享密钥加密一个随机数，服务器解密后返回结果，以确认双方共享相同的密钥。在双向鉴别中，双方互相加密和解密随机数，以确保双方身份的真实性。这种方法的优点是速度快，但密钥管理较为复杂。

2、数字签名技术：数字签名使用非对称加密算法，通常包括一个公钥和一个私钥。发送方使用私钥对消息进行签名，接收方使用公钥验证签名的真实性。数字签名不仅可以确保消息的完整性，还可以提供不可否认性，即发送方不能否认曾发送过该消息。这种方法广泛应用于电子邮件、电子商务等领域。

3、密码校验函数：密码校验函数（MAC）是一种基于对称密钥的算法，用于验证数据的完整性和真实性。发送方使用共享密钥和消息生成一个 MAC 值，接收方使用相同的密钥和消息生成 MAC 值并进行比较。如果两个 MAC 值相同，则消息未被篡改。这种方法的优点是计算效率高，适用于需要高效数据完整性验证的场景。

4、零知识技术：零知识证明是一种密码学技术，允许一方（证明者）在不泄露任何秘密信息的情况下向另一方（验证者）证明其身份。零知识证明的一个经典应用是身份验证，证明者可以通过一系列交互式协议证明其拥有某个秘密，而不泄露该秘密的具体内容。这种方法的优点是安全性高，适用于需要高度保密的场景。这些认证方式通过使用时间戳、序号或随机数等时变参数，防止重放攻击和多次接受先前有效的鉴别信息，从而提高了系统的安全性和可靠性。

2.1.1.1. 经典基于签名算法认证方式

l976 年，Whitfiele Diffie 和 Matin Hellman 首次提出数字签名的概念后，数字签名得到发展，许多数字签名方案被提出。它被广泛的应用于各种密码方案中，以实现数据认证性、完整性和不可否认性。数字签名体制一般以公钥密码为基础，使用用户私钥对信息进行签名，任意验证者通过用户的公钥进行验证，验证通过则说明信息完整，且用户不可否认。数字签名不同于电子签名，数字签名是对所有数据进行签名，主要是保障了数据在传输过程中进行身份认证，保障数据完整不可篡改。由于数字签名的这种特性，在许多行业中得到了广泛的应用。如软件下载过程中，通过验证数字签名可以保障用户下载的软件为没有被篡改的原软件；常见的数字证书(CA)通过对公钥进行签名从而实现身份的认证；金融以及旅游等行业通过数字签名进行合同文件的签署，保证了文件的有效性与完整性。

公钥数字签名在现代通信传输中发挥着重要的角色，如电子投票、数字人民币等，以保障消息机密性、完整性和不可否认性。基于公钥密码的数字签名在早期大部分是基于数论中的困难问题来构建，如基于大整数分解困难假设设计的

RSA 算法，基于离散对数困难假设设计的 ElGamal 算法。但是随着量子计算机的壮大， 这些典型数字签名算法的安全性已经不足以抵抗量子计算机的攻击。如果运用 "Shor 算法"，基于数论的困难问题将不再困难，典型密码算法很容易被攻破。如果只通过延长算法中参数的比特长度来保障其安全性，那么计算效率就会大打折扣，因此，基于公钥密码的数字签名缺乏持久的安全性。

2.1.1.2. 经典基于口令认证方式

1992 年，Bellovin 和 Merrittin [18] 首先提出了基于口令的认证密钥交换(Password-based Authenticated Key Exchange，PAKE)协议，并提出了著名的加密密钥交换协议，简称为 EKE 协议。在 EKE 协议中，两个参与方执行 Diffie-Hellman 密钥交换协议的加密版本，即使用双方共享的口令作为对称密钥对每一条消息进行加密。此后 EKE 协议成为后续许多工作的基础，由于缺少合适的安全模型对 PAKE 协议的安全性进行分析，这些协议只具有启发式的安全性。2000 年，Bellare 等人[19]首先提出了两方口令认证密钥协商协议的形式化安全模型，简称为 BPR 模型，并在理想假设下证明了 EKE 协议变体的安全性，例如理想密码模型(Ideal Cipher Model)和随机谕言(Random Oracle，RO)模型。2001 年，Goldreich 和 Lindell 首先提出了标准模型下可证安全的 PAKE 协议[20]，该协议不依赖于任何可信假设，具有十分重要的理论意义。该协议的通信、计算和轮复杂度较高，因而缺乏实用性。

2.1.1.3. 经典基于令牌认证方式

基于令牌的认证方式是一种现代且安全性较高的身份验证方法，通过分发和验证令牌来确认用户身份。令牌（token）通常是一个随机生成的字符串，用户在登录时通过用户名和密码获取，随后在访问受保护资源时使用此令牌进行验证。这种方式增强了安全性，因为令牌通常有时效限制，即使被截获也只能在短时间内使用，而且令牌的生成和验证过程通常涉及加密技术，进一步提高了安全性。基于令牌的认证方式被广泛应用于单点登录（SSO）、API 访问控制和移动应用中，因其可以减少用户重复输入密码的次数，同时也避免了传统基于会话的认证方法中的一些安全风险。常见的令牌类型包括 JSON Web Token (JWT)、OAuth

令牌和 SAML 令牌等，它们在不同的应用场景中发挥着重要作用，通过提供一种灵活且可扩展的认证机制，有效保护用户数据和系统安全。

2.1.1.4. 经典基于生物特征认证方式

基于生物特征的认证方式是一种利用个体独特的生理或行为特征进行身份验证的现代技术，通过采集和分析指纹、虹膜、面部特征、声音、掌纹、静脉图、步态等生物特征来确认用户身份。与传统的密码和令牌不同，生物特征具有唯一性和难以伪造的优势，极大地提高了安全性和便利性。用户只需简单地通过扫描或识别其生物特征，即可快速完成身份验证，避免了记忆密码或携带物理令牌的麻烦。生物特征认证广泛应用于智能手机解锁、金融交易、门禁系统、机场安检等领域，因其高效性和可靠性，逐渐成为现代身份验证的主流方式之一。然而，这种技术也面临一些挑战，如隐私保护问题、设备精度和环境影响等，需要通过严格的安全措施和技术手段来保障生物特征数据的安全和用户隐私。

2.1.1.5. 经典基于多因子认证方式

基于多因子认证（Multi-Factor Authentication, MFA）是一种增强安全性的身份验证方法，通过结合两种或多种不同类型的验证因素来确认用户身份，从而显著提高系统的安全性。这些验证因素通常分为三类：用户知道的东西（如密码或 PIN 码）、用户拥有的东西（如智能卡、手机或硬件令牌）以及用户自身的生物特征（如指纹、面部识别或虹膜扫描）[21]。MFA 的实施能够有效减轻由于单一验证因素被攻破而导致的安全风险，因为即使攻击者获得了用户的密码[22]，他们仍然需要通过额外的验证因素才能成功访问系统。多因子认证广泛应用于金融服务、企业网络安全、在线服务和云计算平台等领域，成为防范身份盗窃和未经授权访问的重要手段。通过集成多种验证方式，MFA 提供了一层额外的保护，确保只有经过多重验证的用户才能访问敏感信息和资源，这在现代网络安全环境中显得尤为重要。

2.1.2. 经典密钥交换协议

Diffie 和 Hellman 提出了著名的 Diffie-Hellman 密钥交换算法[7]，这一思路为后续公钥密钥交换协议的发展奠定了基础。1984 年，Shamir 提出了身份基础

密码学的概念[2]，为基于身份的密钥交换协议开辟了新的方向,如 Sakai-Ohgishi-Kasahara (SOK)身份基础密钥交换方案[23]。1999 年，SPEKE (Simple Password Exponential Key Exchange)协议[24]利用预共享密码确立会话密钥,提供了抗字典攻击的机制。此外，2008 年，HMQV (Hashed Menezes-Qu-Vanstone)协议[25]提出了一种高性能的安全 Diffie-Hellman 密钥交换机制。这些经典密码算法支撑的密钥交换协议,为安全通信提供了可靠的密钥确立手段，反映了密钥交换技术的不断进步。

2.1.3. 小结

经典认证密钥交换协议是现代网络安全中至关重要的一部分，其主要目标是保证通信双方在不安全的网络环境中安全地交换加密密钥。传统的经典认证密钥交换协议通常依赖于公钥基础设施（PKI）和对称加密技术，广泛应用于电子商务、虚拟私人网络（VPN）等领域。

报告回顾了几种典型的经典认证密钥交换协议，如 Diffie-Hellman 协议、RSA 协议和基于椭圆曲线的协议等。通过这些协议，通信双方能够在不共享任何事先密钥的情况下，通过交换公开信息来共同生成一个共享的秘密密钥，该密钥随后用于加密后续通信。这些协议的安全性主要依赖于一些数学难题，如离散对数问题和整数分解问题，理论上它们在经典计算机下是难以破解的。

然而，经典认证密钥交换协议也存在一定的局限性。随着计算能力的提升和新型攻击手段的出现，这些协议在某些情况下可能遭遇安全性挑战，尤其是面对量子计算的威胁。尽管如此，经典认证密钥交换协议依然是当前网络安全架构中的基础组成部分，且其安全性和效率经过多年的实践验证，具有较高的成熟度和应用稳定性。

2.2. 抗量子认证密钥交换协议

近年来，研究人员针对抗量子认证密钥交换协议进行了大量研究。其中，基于格、编码、同源函数和多变量等抗量子密码算法的认证密钥交换协议备受关注。这些协议利用抗量子密码算法实现身份认证和密钥协商功能，在不降低效能的情况下，提供量子安全性。认证密钥交换协议是基于密码算法完成的，当前的密码算法主要有基于格密码算法、基于哈希的密码算法、基于编码的密码算法等。目前这些抗量子密码算法都在使用。目前该研究领域主要使用基于格的签名/密钥

交换算法、基于哈希的签名算法、基于签名的密钥交换算法等。本报告重点介绍基于格密码的签名和密钥交换密码算法。

格密码学算法如 NTRU[26]和 Kyber[27]，基于格问题的困难性来提供抗量子的密钥交换和数字签名方案。编码密码学算法如 McEliece[28]和 Niederreiter[29]，则利用编码理论中不可逆转的线性码问题作为安全基础，构建抗量子攻击的公钥密码系统。多变量密码学算法如 RAINBOW[30]和 QUARTZ[31]，则依托多元二次方程组求解的困难性来实现公钥密码和数字签名功能。哈希密码学算法如 SPHINCS[32]和 XMSS[33]，使用抗量子安全的密码学哈希函数作为基础，提供抗量子的数字签名。此外，同态加密算法如 FHE[34]和 LWE 同态加密[35]，能够在密文上直接进行计算，为构建抗量子安全的计算方案提供了新思路。这些新兴的后量子密码技术为保护通信和计算安全性带来了新的解决方案。

其中，本报告主要研究基于格密码的抗量子密码协议。基于格理论构造密钥交换的思想最早由 Katz 等人提出， 他们给出了新的基于格的选择密文攻击安全的公钥加密以及第一个格上平滑投射哈希函数，并构造了基于格的认证密钥交换协议。研究人员对基于格的 AKE 方案进行了深入研究，将认证方案和抗量子密钥交换协议相结合，实现通信双方的身份认证和通信数据的机密性保护。

2005 年，Regev 提出了基于格的学习带误差问题（LWE），为后续的研究奠定了基础[36]。2011 年，Lyubashevsky 等人提出了基于环上学习带误差问题（RLWE）的密钥交换协议[37]。近年来，多个基于格的认证密钥交换协议相继提出。2015 年，Alkim 等人提出了 NewHope 协议，该协议在提高安全性和效率方面取得了显著进展[38]。2017 年，Bos 等人提出了 Kyber 协议，该协议基于模块格，进一步优化了密钥交换的性能[39]。2019 年，Bos 等人提出了 FrodoKEM 协议，该协议基于标准格，提供了更高的安全性[40]。2021 年，Ren 等人提出了基于 MLWE 的口令认证密钥交换协议，实现通信双方的身份认证和通信数据的机密性保护。基于 MLWE 设计方案，提供了通信双方多种参数选择保证了在多种场景中也可以应用[41]。

2.2.1. 抗量子认证方式

抗量子认证方式和经典的认证方式相似，包括：基于签名算法的认证和基于令牌的认证等。基于签名算法的认证使用抗量子签名算法，如基于格的签名算法，来确保数据的完整性和真实性。基于令牌的认证则利用抗量子令牌，通过生成和验证抗量子令牌来进行身份验证。每种认证方式都旨在抵御量子计算对传统密码系统的威胁，确保信息安全。

2.2.1.1. 基于抗量子签名算法认证方式

基于格的数字签名主要有两种构造签名方案，一种为 Hash-and-Sign 签名，另外一种为 Fiat-Shamir 签名。Hash-and-Sign 签名首先会对信息 m 进行 hash 运算，然后对 hash(m)进行签名，验证的时候需要对 hash(m)进行验证。典型的 Hash-and-Sign 签名为 GPV 方案，存在效率低下等问题，适合用于计算资源受限的设备。NIST 新标准之一 falcon 方案就使用了 GPV 框架，使用多种技术提高了签名的效率，具有紧凑性、带宽需求小、验签速度快与签名尺寸小等优势，它的设计考虑了嵌入式设备的资源限制，能够再内存受限的环境中高效运行。Fiat-Shamir 签名使用了随机预言机生成非交互式签名方案，此模式相对于 Hash-and-sign 具有简单高效的特点，受到人们的喜欢，许多数字签名方均是在这个模式基础上进行设计的。在基于格实现的 Fiat-Shamir 签名中，由于格的特殊代数结构，如果直接输出算法的签名，会泄露密钥，为解决这个问题，VadimLyubashevsky 提出“异常终止”的概念，因此基于 Fiat-Shamir 签名引进了拒绝采样技术，使得输出满足某种概率分布，从而保护了密钥的安全。

2.2.1.2. 基于口令的抗量子认证方式

目前已经有很多基于格的密钥交换协议融入了基于口令的认证方式，提出了 PAKE 协议。在 2015 年，Jarecki 等人提出了基于格的 PAKE 协议，利用学习带误差（LWE）问题实现抗量子攻击的安全性[42]。2016 年，Zhang 等人提出了基于环上学习带误差（RLWE）问题的 PAKE 协议，利用环结构提高了协议的效率和安全性[43]。2018 年，Chen 等人提出了一个基于 LWE 的高效 PAKE 协议，通过优化协议结构和参数设置，显著提高了执行效率[44]。2019 年，Hao 等人提出了一个基于 RLWE 的 PAKE 协议，结合非交互式零知识证明（NIZK）来降低通

信轮次[45]。2020 年，Liu 等人提出了一个在标准模型下具有可证明安全性的 PAKE 协议，避免了随机预言机模型带来的潜在安全威胁[46]。

2.2.1.3. 基于令牌的抗量子认证方式

基于格的令牌认证方式利用格密码学的安全性，通过格签名算法生成和验证令牌，从而提供对量子计算机攻击的抗性。这种方法在身份验证和授权中具有广泛的应用前景。具体来说，格密码学通过使用格签名算法，如 NTRU（N-th degree truncated polynomial ring units）或 Dilithium 等，可以生成安全的令牌。这些令牌在传输过程中难以被破解，从而确保了通信的安全性。此外，基于格的令牌认证方式还可以结合多因素认证（MFA）和基于时间的一次性密码（TOTP）等机制，进一步增强安全性。例如，在多因素认证中，用户不仅需要提供基于格的令牌，还需要通过其他验证手段（如短信验证码或生物识别）来完成身份验证。这种多层次的安全措施可以有效防止未经授权的访问。

2.2.1.4. 抗量子密钥交换协议

2012 年，丁等人提出了第一个错误协调机制[47]设计了一种基于 RLWE 的密钥交换协议。后来，基于 LWE 的 Frodo 协议[48]被提出，它使用 Peikert 的错误协调机制来实现密钥一致性。这项工作在提出一种新的噪声分布方法的同时，也提供了一种动态生成公共噪声的方法矩阵（A）。后来，Joppe W.Bos 等人用 C 语言实现了基于 RLWE 的密钥交换算法[49]。尽管它们的恒定时间 R-LWE 实现比椭圆曲线 Diffie-Hellman 慢 1.8-2.6 倍（ECDH），通过装配级优化可以大大提高效率。2016 年，一个新的密钥协议 col NewHope[50]被提出，它使用 D_4 进行错误协调，减少了降低了故障率，提高了后量子安全性。同时，数论变换（NTT）[51][52]用于加速多项式乘法运算，提高了协议的运算效率。2018 年，Jan Pieter D'Anders 等人提出了基于 LWR 模块的 Saber。他们提出使用 Toom-Cook 和 Karatsuba 多项式多重乘法的组合来代替 NTT 算法，并提供了一个高级常数时间软件实现[53]。他们还提出了 CCA 安全（选择密文攻击安全）密钥封装机制。同时，Joppe-Bos 等人提出了一个高度优化的实例化被称为 Kyber[54]。他们应用了 Fujisaki–Okamoto 变换的变体[55]创建 CCA 安全的 KEM Kyber。基于 MLWE 问

题，它在确保安全性的同时具有灵活性，安全性可以通过以下方式增加或减少改变矩阵的维数 k。2020 年，Wang 等人[56]提出了使用 Saber 加速多项式乘法的改进实现大整数协处理器。Joppe-Bos 等人[57]使用了 Kronecker+算法加速环中多项式乘法并集成算法在 Saber[54]中，我们展示了 Kronecker+的实用性。由于之前的协议中缺乏全面的病房安全，2021 年，Boyd 等人[57]构建了一个基于预共享密钥的轻量级认证密钥交换协议。它实现了完全的前向安全性，仅依赖于简单高效的对称密钥原语。将协议应用于计算能力有限的设备和存储容量，在 2023 年，Zhao 等人[58]提出了一种基于点阵的密码认证密钥交换协议。他们使用近似平滑的投影哈希函数和密钥共识来设计非对称密钥协商结构，使设备能够实现密钥交换，同时存储更少认证信息。

2.2.2. 小结

本报告深入研究了抗量子认证密钥交换协议的现有成果与发展趋势。随着量子计算技术的崛起，传统的基于经典密码学的密钥交换协议面临着巨大的安全威胁，因此，设计能够抵抗量子计算攻击的密钥交换协议成为了密码学研究中的重要课题。

上述内容回顾了几种主流的抗量子认证密钥交换协议，并对它们的安全性和性能进行了分析。具体来说，这些协议主要基于格密码、哈希函数、同态加密等密码学原理，旨在为未来量子计算环境下的安全通信提供保障。尽管现有的抗量子协议在理论上能够有效抵御量子计算的攻击，但在实际应用中，它们仍面临计算复杂度较高、通信开销大等问题。

总体来看，抗量子认证密钥交换协议的发展仍处于不断探索和完善的阶段。未来的研究需要在保证抗量子安全性的基础上，进一步优化协议的效率，降低其资源消耗，以实现在网络协议中的应用。

2.3. 混合抗量子认证密钥交换协议

随着量子计算机的发展,传统的密码算法可能会被破解,因此研究抗量子攻击的密码协议成为当前密码学领域的一个重要方向。而纯量子密码协议实现成本高、操作复杂,因此出现了将经典密码算法和量子密码算法结合的混合量子密码协议。这类协议在设计架构、安全性分析和性能评估等方面成为研究重点。目前已有多

种混合量子密码协议被提出,如 google 提出的抗量子签名算法、IETF 提出的针对 TLS 协议的修改草案以及 Paul 等人在 2021 年提出的 TLS 协议的混合认证方式等,并在实验中验证了可行性。未来的研究将继续优化这些协议的性能指标,探索在更多应用场景中的使用,并进一步降低成本和复杂度,提高实用性。总的来说,混合量子密码协议为应对未来量子计算机威胁提供了一种可行的解决方案,是当前密码学领域的一个重要研究方向。

2.3.1. 混合认证方式

目前混合认证方式中在混合签名方案中研究最多，以下主要研究混合签名方案。在 2023 年，Nina 等人提出并实现了证明可组合性（Proof Composability）、SNS 和同时验证（Simultaneous Verification）（下文将其组合称为混杂）[59]。他们给出了几种基于 Fiat-Shamir 签名方案（如 Dilithium）或 Falcon 与 RSA 或 DSA 的组合。并观察到，有许多签名混合目标，其中一些通过并行签名或串联方法无法实现。这些目标包括证明可组合性（即后量子混合签名的安全性可以轻松链接到组件算法）、弱可分离性、强可分离性、向后兼容性、混合通用性（即一旦被证明是安全的，可以用不同算法实例化的混合组合）和同时验证。其展示了抗量子混合认证方式在网络协议中的可行性。

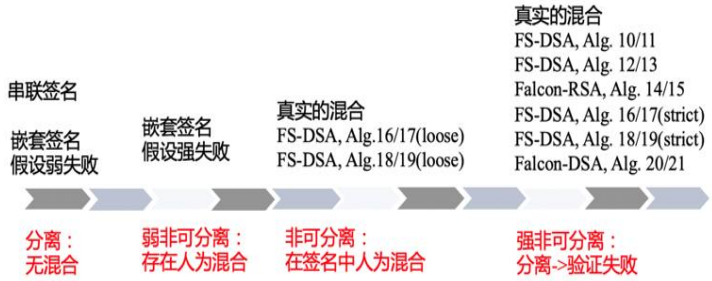


图 1（非）分离混合签名方案

非可分离性表示可分离性程度的尺度。在图 1 的最左端是那些其中一个成分签名可以被剥离而验证者在验证过程中无法检测到变化的方案。主要是由简单的签名串联组成。嵌套签名（其中消息由一个组件算法签名，然后消息-签名组合由第二个组件算法签名）也可能属于这一类别，这取决于内层或外层签名是否被“剥离”。在弱混合失败的情况下，如果外层签名被剥离，混合签名的任何痕迹可能都不会保留（取决于各种实现细节）。

在弱非可分离性下，如果一个混合签名的复合签名被移除，混合的痕迹将会保留。这使得验证者能够检测到组成签名从混合签名中被剥离。例如，在一个串联的 ECDSA-Dilithium 混合中，需要一个经过认证的 ECDSA-Dilithium 混合公钥组合，ECDSA 组件公钥可以用于验证仅包含 ECDSA 组件签名。在这种情况下，检测取决于认证混合公钥的实现要求的“痕迹”。这说明混合签名是否可分离或（弱）不可分离也可能取决于协议实现的层面，即超出算法级别的安全方面，例如 EUF-CMA 之外的标准定义。嵌套签名提供了另一个例子，在强混合失败下，验证者可能被欺骗，将一个新消息解释为消息/内部签名组合，并仅验证外部签名。在这种情况下，内部签名标签也是一个“痕迹”。

在该规模上的第三个概念是非可分离性概念，其中可分离性检测依赖于签名本身的痕迹。与弱非可分离性不同，其中痕迹可能在实际消息、证书或其他非签名组件中，这个概念更密切地与传统的算法安全概念（如 EUF-CMA）相关，其中安全性取决于签名算法的内部结构和验证过程。在这种类型中，验证者能够在算法级别的验证过程中检测到痕迹。

在最强的非可分离性概念的方案中，当其中一个组成签名缺失时，验证不仅失败，而且如果验证者无法验证两个签名，验证也会失败。这种非可分离性构造最接近传统的数字签名，其中假设验证者确实验证了一个签名，结果要么是对完整签名的肯定验证，要么是如果签名无效则失败。对于混合签名，一个“完整签名”意味着两个组件算法的存在，因此最强非可分离性概念对验证采取了全有或全无的方式。因此，经历了任何分离攻击的更可分离的签名都无法实现混合验证。

2.3.2. 混合密钥交换方式

在混合密钥交换协议的研究中，目前学术界已经开展了很多网络协议的应用。IETF 提出了多种混合密钥交换方式在 TLS 中的应用。

2.3.2.1. 抗量子安全的混合加密方法在 TLS 协议中的应用[60]

在 2017 年，互联网工程任务组（Internet Engineering Task Force）提出了一种对于 TLS 的修改方式。提供了一种模块化设计，允许在握手协议中采用一个或多个量子安全的加密系统，适用于 TLS 版本 1.0 到版本 1.3 [RFC2246]，[RFC4346]，[RFC5246]，[TLS1.3]。使用了一个混合方法，将经典的握手机制与

使用量子安全加密方案实例化的密钥封装机制相结合。这种模块化设计通过引入仅一个新的密码套件，为 TLS 提供了量子安全的功能。同时，它具有灵活性，可以包括当前和将来的新型量子安全加密方案。在 TLS 中包含新的量子安全密钥交换算法，同时保持使用已建立的密码套件所带来的保障。它允许使用已建立的经典密码套件和量子安全的密钥封装机制来协商 TLS 的预主密钥。

Client C		**Server** S
ClientHello*	→	
		ServerHello*
		Certificate*
		ServerKeyExchange*
		CertificateRequest*
	←	ServerHelloDone
Certificate*		
ClientKeyExchange*		
CertificateVerify*		
ChangeCipherSpec		
Finished	→	
		ChangeCipherSpec
	←	Finished
Application Data	←→	Application Data

图 2 完整的 TLS 1.2 握手的消息流程

图 2 显示了 TLS 密钥建立协议（也称为完整握手）涉及的所有消息。量子安全密码学的添加仅对 ClientHello、ServerHello、ServerKeyExchange 和 ClientKeyExchange 产生直接影响。在本文档的其余部分，我们将更详细地描述每个量子安全密钥交换数据结构，包括这些消息的内容和处理方式。身份验证由经典密码学提供。引入量子安全加密方案提供了对抗量子攻击者的前向保密性。客户端和服务器之间交换的附加密码数据如图 3 所示。

Client C		**Server** S
ClientHelloExtension		
(SerPKReq|		
QSHSchemeIDList|		
CliPKList)	→	
		ServerHelloExtension
		ServerKeyExchange
	←	(SerCipherList)
ClientKeyExchange*		
(CliCipherList)	→	
ClassicS|SerS|CliS	←→	ClassicS|SerS|CliS

图 3 握手过程中的额外数据交换

ClientHello 消息包括客户端希望协商的经典密码套件列表（例如 TLS_ECDH_ECDSA_WITH_NULL_SHA），以及一个新的密码套件标识符 TLS_QSH（代表带有量子安全混合握手的 TLS）。这个新的标识符应该在密码套件列表中首先出现。

1. ClientHelloExtension 中包含了三个新增的域

（1） SerPKReq：请求服务器公钥的消息；同时指示客户端希望接收的服务器公钥数量。

（2） QSHSchemeIDList：客户端提供了一个不同的 QSHSchemeID 列表，每个 ID 代表客户端支持的量子安全加密方案/参数集。

（3） CliPKList：客户端提供了一个公钥列表，格式为[CPK1]||[CPK2]|...，其中每个公钥与 QSHSchemeIDList 中的不同 QSHScheme 相对应。

客户端并不需要为列表中的所有 QSHScheme 提供公钥。客户端可以通过省略相应的公钥字段来表示不希望为某些 QSHScheme 贡献公钥。

2. ServerHelloExtension 中包含了一个新增的域

SerPKList：一个服务器公钥列表，格式为[SPK1]||[SPK2]|...，其中每个公钥与 QSHSchemeIDList 中的 QSHScheme 相对应。

SerPKList 是一个由公钥结构 QSHPK 组成的列表。对于每个结构，都有一个 QSHSchemeID 字段用于标识对应的加密方案。这个 QSHSchemeID 必须存在于 QSHSchemeIDList 中。

3. ServerKeyExchange 中包含一个新增的域

SerCipherList：密文列表，格式为：

[Encrypt_CPK1(SerS1)]||[Encrypt_CPK2(SerS2)]|...，其中服务器贡献的秘密密钥材料为 SerS = SerS1|SerS2|...，而 CPKi 是从 CliPKList 中选择的。

SerPKList 和 SerCipherList 中至少必须有一个至少有一个条目。

此外，ServerKeyExchange 包含对所选的经典密码套件的指示，以及适用于该密码套件的 ServerKeyExchange 材料。

如果提供了 SerPKList，则 ClientKeyExchange 消息必须包含一个额外的密文列表。

4. CliCipherList：密文列表

[Encrypt_SPK1(CliS1)]|[Encrypt_SPK2(CliS2)]|...，其中客户端贡献的秘密密钥材料为CliS = CliS1|CliS2|...，而SPKi来自SerPKList。客户端必须使用SerPKList中的所有公钥。此外，ClientKeyExchange包含适用于所选经典密码套件的ServerKeyExchange材料。SerS和CliS不能同时为空。客户端和服务器协商的最终预主密钥是经典预主密钥、SerS和CliS按照顺序连接而成的。在握手结束时，使用经典密码套件指定的伪随机函数（参见RFC 5246 [RFC5246]第8.1节）从预主密钥派生出一个48字节的固定长度主密钥。

2.3.2.2. 混合抗量子密钥封装方法在 TLS 协议上的应用

在2019年，混合密钥交换的概念被提出，其是指执行两个独立的密钥交换，并将两个生成的共享密钥输入到伪随机函数（PRF）中，目的是派生一个与两个密钥交换中较强的密钥相当安全的密钥。本文档描述了适用于传输层安全性协议1.2（TLS）的新混合密钥交换方案。这些密钥交换方案基于将ECDH与抗量子密钥封装方法（PQ KEM）结合使用现有的TLS PRF。具体而言，本文档指定了在TLS 1.2握手中将比特翻转密钥交换（BIKE）和超奇异同源密钥交换（SIKE）方案与ECDHE结合使用作为混合密钥协商的方法。

在此方案中以支持PQ混合密钥交换，适用于TLS版本1.2 [RFC5246]。这些扩展旨在支持NIST提案征集中的大多数第二轮候选方案。然而，本文档仅为可能的混合密钥协商方法的一小部分定义了密码套件。具体而言，它定义了将ECDHE与BIKE或SIKE结合使用作为混合密钥协商方法的方式。

使用ECDHE与BIKE或SIKE来计算TLS的预主密钥。主密钥的派生增加了包含ClientKeyExchange消息的步骤。加密/MAC密钥和初始化向量的派生与密钥交换算法无关，并不受引入这些混合密钥交换的影响。虽然只定义了使用PQ KEM混合密钥交换与BIKE或SIKE的用法，但它特别设计成可以轻松扩展以包括其他的PQ KEM方法。

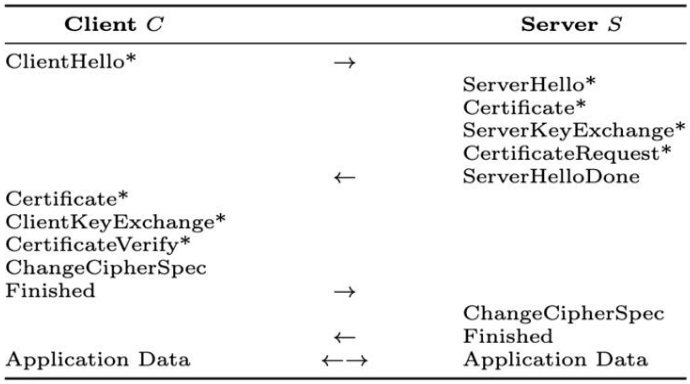


图 4 混合 TLS 协议中的信息流

图 4 显示了 TLS 密钥建立协议（也称为完整握手）中涉及的消息。混合密钥交换的添加直接影响了 ClientHello、ServerHello、ServerKeyExchange 和 ClientKeyExchange 消息。在接下来的部分，[KEM]指的是 BIKE 或 SIKE。服务器在 ServerKeyExchange 消息中发送其临时 ECDH 公钥以及使用相应曲线和[KEM]参数生成的临时[KEM]公钥。本规范要求这些参数必须使用与服务器证书中的公钥相对应的签名算法进行签名。客户端在与服务器的临时 ECDH 密钥相同曲线上生成 ECDHE 密钥对，并根据服务器提供的[KEM]公钥计算密文值，并将它们发送到 ClientKeyExchange 消息中。客户端计算并保留 PQ KEM 封装密钥（K）作为预主密钥的一部分贡献。客户端和服务器都执行 ECDH 操作，并将生成的共享秘密（Z）作为预主密钥的一部分。服务器计算 PQ KEM 解封装例程以计算封装密钥（K），或在解封装失败时生成错误消息。将 ECDHE 和[KEM]密钥合并为 TLS 1.2 [RFC5246]预主密钥的机制。在混合密钥交换中，服务器和客户端都计算出两个共享秘密：来自 RFC 8422 的先前定义的 ECDHE 共享秘密 Z，以及来自底层 PQ 密钥封装方法的另一个共享秘密 K。形成 ECDHE_[KEM]混合密钥交换的预主密钥的方法是将 ECDHE 共享秘密 Z 与 KEM 密钥 K 连接在一起，形成不透明数据值 premaster_secret = Z || K。

2.4. TLS 协议抗量子密码计算密码迁移研究现状

目前，世界各主要大国都针对抗量子密码算法及其迁移工作进行了研究。世界各主要大国在 PQC 标准方面均以美国 NIST 的标准为风向标。欧盟、日本、加

拿大、澳大利亚、新西兰等甚至准备直接采用 NIST 的 PQC 标准。而其他国家的 PQC 标准也基本上是从 NIST 的全球征集活动当中延伸而来。NIST 在经过数十年的研究以及密码算法的征集工作，于 2024 年 8 月发布了三个抗量子密码算法的标准，分别是 FIPS203《Module-Lattice-BasedKey-EncapsulationMechanismStandard》[61]、204《Module-Lattice-BasedDigitalSignatureStandard》[62]和 205《StatelessHash-BasedDigitalSignatureStandard》[63]，为抗量子密码算法的应用提供了基础。在算法系列标准发布后，其应用和传统密码协议的抗量子迁移成为目前的研究热点。

美国 NIST 下属国家网络安全卓越中心 NCCoE 于 2018 年启动产业化迁移工作，并于 2023 年 4 月发布第一份迁移准备工作报告。美国迁移工作由白宫亲自领导，于 2022 年 4 月发布国家安全备忘 NSM-10，要求 2035 年完成抗量子密码迁移工作。美方的迁移工作涉及白宫管理与预算办公室、NIST、国家安全局、国防部等多个政府机构，以及亚马逊、思科、戴尔、IBM、微软、JP 摩根等 IT 和金融领域领军企业。IBM 参与开发 CRYSTALS-Kyber 算法[64]成为 NIST 第四轮 PQC 标准候选算法，并设计基于 CRYSTALS-Kyber 算法的 TLS 原型。其研究人员深度参与 NIST 后量子密码标准化流程，推动 Kyber 成为 TLS 等协议的核心选项，尽管其密钥生成效率较传统 ECDHE 仍存在约 40%的延迟瓶颈。谷歌在 Chrome116 版本(2023 年 8 月)中率先引入 X25519Kyber768 混合密钥交换协议，结合经典椭圆曲线算法 X25519 和 NIST 标准抗量子算法 Kyber-768。该协议通过“双重加密”机制生成会话密钥，抵御量子计算机的“现在截获，未来解密”攻击。在兼容旧版本协议的同时将握手延迟控制在 30%以内的增幅。亚马逊 AWS 作为云计算服务商，积极推动 NIST 标准算法(如 CRYSTALS-Kyber)的落地测试，其 AWS 云服务为后量子算法的性能评估提供了大规模算力支持。AWS 参与提交了 BIKE 算法[65]和 SIKE 算法[66]，并对其效率进行测试，发现采用 BIKE 与 X25519 混合的方案会比传统的方案慢 20ms，采用 SIKE 与 X25519 混合的方案会慢 150ms(传统方案握手耗时 1.19ms)。AWS 团队在 2022 年 7 月在其设计的 s2n-tls 协议中提供 CRYSTALS-Kyber 算法的支持，并在 2023 年 1 月不再支持 SIKE 算法。Meta 主导的 OpenQuantumSafe(OQS)开源项目[67]，通过 LibOQS 密

码库实现抗量子算法与 TLS1.3 的模块化集成，为学术界和中小企业提供了标准化测试平台，并在实际部署中发现 TCP 和 TLS 协议混用产生的问题。三星推出的 S3SSE2A 芯片是业界首个集成硬件后量子密码学(PQC)的安全芯片，支持移动设备关键数据的抗量子加密。该芯片通过硬件加速优化后量子算法(如 NIST 标准算法)的执行效率。

欧洲抗量子密码产业迁移工作主要由 ETSI 协调推进，自 2016 年至今已发布十余份研究、评估、指导报告，从算法、技术、标准、服务、市场五个层次全面展开，并在移动通信、金融、智能交通等领域率先进行示范工作。加拿大滑铁卢大学的 OpenQuantumSafe 项目已经针对 TLS、SSH、X.509、CMS、S/MIME 等典型安全协议进行了集成测试，吸引了国际一流 IT 企业的参与，是目前国际上最具影响力的抗量子密码测试平台。美国 JP 摩根于 2022 年牵头负责美国金融领域的抗量子密码迁移工作，国际清算银行于 2023 年启动了抗量子密码迁移示范工作。2023 年 11 月，我国金融行业启动了《银行业及其关键基础设施信息系统的抗量子密码迁移技术研究》项目，以推进国内金融领域的抗量子迁移工作。

在 TLS 握手协议中，客户端和服务端首先在不安全的通信信道上使用密钥交换协议来协商出用于后续会话加密的共享密钥，以确保后续会话在安全的通信信道中进行。在 TLS 握手协议中，用于密钥协商的算法是(EC)DHE 协议，它是由 Diffie 和 Hellman 在 1976 年提出的 Diffie-Hellman 密钥交换协议的演变版本。

(EC)DHE 协议使得通信双方能够通过协商确定后续会话所使用的对称密钥。在 TLS 协议的握手流程中，(EC)DHE 协议中的末尾字母 "E" 表示 "ephemeral"，即每次协商时通信双方都会生成随机的私钥，这些私钥具有一次性的特性，彼此独立。这使得(EC)DHE 协议具备前向安全性，即过去的会话不会被未来的攻击者所破解。然而，它并不具备抵抗量子攻击的能力。因此，构建和实现基于抗量子密码的 TLS 协议被认为是抗量子密码领域的一个重要研究方向。这样的研究旨在设计 TLS 协议的抗量子密码版本，以应对未来量子计算机对传统加密算法的威胁。这将确保 TLS 协议在量子计算环境下依然能够提供安全的通信保护。

随着量子计算的发展，基于抗量子密码算法的 TLS 协议的研究已经取得了许多学术成果。目前，这方面的研究主要集中在以下两个方向上：第一种方向是

将抗量子密码算法添加到现有的 TLS 草案或规范中，但尚未进行具体实现和性能评估。这种研究探索了如何在 TLS 协议中集成抗量子密码算法，以应对未来量子计算机对传统加密算法的威胁。第二种方向是设计抗量子密码算法与 TLS 协议相结合，并对其进行性能评估。这些研究旨在开发新的抗量子密码算法，使其与 TLS 协议相兼容，并评估其在实际应用中的性能表现。这包括算法的安全性、计算和通信开销以及与现有 TLS 实现的兼容性等方面的评估。

这些研究方向的目标是为后量子时代提供安全可靠的通信保护，确保 TLS 协议在量子计算环境下仍能有效地保护通信的机密性和完整性。这些努力对于确保互联网通信的安全性和隐私保护具有重要意义。

在第一种研究方向中，已经有一些相关草案和规范被提出。2016 年，IETF（Internet Engineering Task Force）制定了将 NTRU 密码体制融入 TLS 1.2 协议的握手流程中的规范[68]。2019 年，IETF 制定了将比特翻转密钥交换协议（Bit Flipping Key Encapsulation， BIKE）和超奇异同源密钥交换协议（Supersingular Isogeny Key Encapsulation， SIKE）与基于椭圆曲线的 Diffie-Hellman 密钥交换协议相结合，并作为 TLS 1.2 握手协议的混合密钥协议[69]。同时，IETF 还对在 TLS 1.3 协议中使用混合密钥交换的各种设计进行了分类，这是考虑到向抗量子密码学过渡的需要。这些草案和规范的提出旨在探索在 TLS 协议中集成抗量子密码算法的方法和机制，以应对未来量子计算机对传统加密算法的威胁。这些努力的目标是确保 TLS 协议在量子计算环境下仍能提供安全的通信保护。

第二种研究方向产生了一些学术成果：2015 年，Bos 等人[70]在 TLS 1.2 协议的应用场景下设计了一种基于 RLWE 问题的密钥交换协议。他们通过这种协议，使得 TLS 1.2 协议在握手尺寸增加的情况下，仅减少了 21%的吞吐量，并达到了 128 位的安全级别。2016 年，Bos 等人[71]基于容错学习问题（Learning with Errors， LWE）提出了 Frodo 算法，并将其应用于 TLS 1.2 协议。相比基于 RLWE 问题的 TLS 1.2 协议，该算法的通信开销增加，但吞吐量减小程度较小，延迟增加也较少，仍然具备一定的优势。2017 年，Stebila 等人讨[72]论了 BCNS15 和 Frodo 两个协议在 TLS 协议背景下的安全性，并将抗量子算法库 liboqs 整合到 TLS 1.2 协议中。同时，高昕炜等人将基于 Ziggurat 采样算法的离散高斯采样器

应用于基于 RLWE 问题的认证密钥交换协议，并提出了抗量子 TLS 密码组。2018 年，高昕炜等人[73]构建了一种基于 RLWE 问题的随机化密钥交换协议，该协议能够抵抗密钥重用攻击。同时，他们还构建了一个抗量子 TLS 密码组，其中包含更高效的基于 RLWE 问题的认证密钥交换协议。2019 年，Kwiatkowski 等人[74]针对不同密钥大小对 TLS 协议握手时间的影响进行了研究，并将基于同源的 SIKE 方案和基于 NTRU 的 HRSS17 方案应用于 TLS 协议中，基于 BoringSSL 进行了性能测试。此外，Langley 与 Cloudflare 合作，将微软的 SIKE 代码集成到 BoringSSL 中进行测试，证明 SIKE 在 TLS 协议中具有一定的优势。2020 年，钟焰涛等人[75]提出将 SWIFFT 函数与 NTRU 签名算法应用于 TLS 1.3 握手协议，使得协议能够抵抗量子攻击。2022 年，张枫等人[76]在 Facebook 的 fizz 库的基础上构建了具备抗量子安全性的 TLS 1.3 协议密码库，并设计了一个能够在任意网络条件下对 TLS 1.3 协议进行性能测试的实验框架。他们对修改后的 fizz 库进行了大规模全面测试。需要注意的是，上述方案中，大多数都将 NTRU 密码体制与基于同源的 SIKE 密码体制应用于 TLS 协议中，而基于格上困难问题的 TLS 1.3 协议研究方案较少。然而，基于同源的密码体制在实际应用中的可行性较低，而 NTRU 密码体制的解密机制还不够完善，可能出现解密错误的情况。因此，在基于格上其他困难问题的 TLS 1.3 协议研究方向上，仍然存在很大的研究空间。

2.5. TLS 协议抗量子迁移存在的问题及挑战

各个国家的组织机构在抗量子密码迁移工作上的部署和进展，既为我国银行金融领域的抗量子密码迁移带来了压力，也提供了经验和方向。目前我国金融领域密码协议迁移主要存在的问题和挑战如下：

问题一：抗量子密钥交换算法和签名算法形成两套体系，且签名算法参数更大，同时迁移抗量子签名算法和抗量子密钥交换算法，尺寸成倍增加，进一步抬高了抗量子迁移改造实施的门槛。受项目排期、硬件换代周期、第三方厂商适配进度制约等因素影响，银行信息系统中各节点抗量子改造推进节奏无法同步，步调难以统一，长期存在多种异构节点共存的复杂通信环境。以数字证书体系为例，手机终端可能已部署了 ML-KEM 抗量子证书，而网银网关、服务系统可能仅升级了抗量子签名证书或者仍使用传统 ECC/RSA 证书。在此差异化改造过渡期

内，异构节点将无法完成握手协商、身份校验与安全会话建立，直接阻断手机银行登录、支付交易、跨行支付等核心金融业务。为此需要建立兼容互认机制，保障新旧体系节点间可正常握手连通。

挑战一：抗量子迁移进度不一致造成节点异构的现状长期存在，需要研究异构节点互通兼容机制。

问题二：后量子抗量子算法的密钥长度、报文参数规模显著高于传统密码算法，现有 TLS 协议均基于传统密码参数设计，原有协议的交互时序、报文分包传输规则、报文传输机制与会话处理逻辑难以承载大容量参数传输。在银行高并发交易、低时延清算、海量终端并发接入场景下，传输大尺寸密钥封装与签名数据时极易出现报文超限、握手往返时延激增等问题。进而频繁诱发密钥协商流程超时、安全会话建立失败、业务连接中途异常断开、交易链路中断等系统性稳定性故障，直接影响金融业务连续性。

挑战二：抗量子算法尺寸膨胀容易导致报文分包，当前分包处理机制不健全易造成握手中断，需设计抗分包 TLS 握手机制。

3. 抗量子 TLS 协议方案设计

针对密码协议面临的量子计算安全威胁，本章结合后量子密码技术发展现状、国际标准化组织的迁移指导建议以及现有密码基础设施改造经验，开展密码协议抗量子迁移方案设计。密码协议的抗量子迁移是一项涉及密码算法、协议机制、系统架构、应用适配和运维管理等多个层面的系统工程，其实施过程需要兼顾安全性、兼容性、性能和可持续演进能力。因此，在开展具体协议迁移设计之前，有必要建立统一的迁移实施策略，对迁移工作的整体流程、实施步骤和关键环节进行规划和指导。

本章以当前应用最广泛的 TLS 协议为研究对象，通过对 TLS 协议基本信息、密码算法应用现状以及迁移节点进行分析，提出面向 TLS 协议的抗量子迁移技术方案，为后续迁移实施与验证提供设计依据。

3.1. 密码协议基本信息调研

<table>
<tr><td colspan="4">协议基本信息表</td></tr>
<tr><td>协议名称</td><td>传输层安全性协议（TransportLayerSecurity）</td><td>协议版本号</td><td>TLS1.3</td></tr>
<tr><td>协议简介</td><td colspan="3">TLS（传输层安全协议，TransportLayerSecurity）是一种广泛应用于网络通信中的加密协议，旨在确保数据在客户端与服务器之间的传输安全。通过对数据进行加密、认证和完整性保护，TLS能够有效防止信息被窃听、篡改或伪造。</td></tr>
<tr><td>金融领域应用场景</td><td colspan="3">在金融场景中，TLS 协议发挥着至关重要的作用。金融机构处理的敏感信息，如交易数据、客户账户信息、身份认证凭证等，均需要高度安全的传输渠道。TLS 通过提供强大的加密机制，确保金融交易的机密性和数据完整性，从而避免潜在的网络攻击和数据泄露。此外，TLS 协议还支持双向认证机制，能够通过客户端和服务器相互验证身份，防止身份伪造与中间人攻击。这种安全特性对于银行、支付系统以及证券交易平台等金融机构至关重要，能够保障交易的安全性、稳定性和可靠性。</td></tr>
<tr><td>密码算法</td><td colspan="3">密钥交换算法：TLS1.3 主要采用椭圆曲线 Diffie-Hellman（ECDHE）和有限域 Diffie-Hellman（DHE）来实现前向安全（PerfectForwardSecrecy，PFS），确保在密钥泄露的情况下，历史通信不会受到影响。常用的椭圆曲线有：
X25519：轻量高效，广泛应用于现代密码学中。
secp256r1：基于 NISTP-256 椭圆曲线标准。
对称加密算法：TLS1.3 仅支持使用经过认证的加密算法（AuthenticatedEncryptionwithAssociatedData，AEAD）。主要使用的对称加密算法有：
AES-128-GCM：带有 Galois/Counter 模式（GCM）的 128 位高级加密标准，提供认证和加密功能。
AES-256-GCM：使用 256 位密钥的 AES-GCM 算法，安全性更高。</td></tr>
</table>

	ChaCha20-Poly1305：一种高效的加密认证算法组合，特别适合在低性能设备上使用。 **哈希算法：**TLS1.3 使用哈希算法来确保数据的完整性和验证数字签名。支持的哈希算法包括： SHA-256：256 位哈希算法，用于消息认证、密钥衍生等。 SHA-384：384 位哈希算法，主要用于加密套件中需要更高安全性的场景。
数字证书	TLS1.3 中的证书主要基于 X.509 标准，常见的证书类型有： **RSA 证书：**尽管 TLS1.3 不再使用 RSA 进行密钥交换，但仍可以使用 RSA 算法来对证书进行签名或验证，常见的有 RSA-PSS 和 RSA-PKCS#1v1.5。 **ECDSA 证书：**采用椭圆曲线数字签名算法（ECDSA）的证书，与 ECDHE 结合可以提供更好的性能和安全性。常用的曲线包括 secp256r1 和 secp384r1。
算法提供密码功能	在 TLS 协议中，密码算法的主要作用是提供安全的数据传输通道。TLS 的密码算法分为四个主要功能类别，每一类对应不同的安全需求，以确保数据的机密性、完整性、身份验证和抗窃听。以下是 TLS 密码算法提供的四种主要功能： 1.密钥交换算法 **功能**:确定客户端和服务器之间的共享密钥，用于后续的加密通信。 **常见算法**: **RSA**:基于非对称加密的密钥交换，但逐渐被弃用。 **DHE（Diffie-HellmanEphemeral）**:提供前向安全性，使用短期会话密钥。 **ECDHE（椭圆曲线 Diffie-HellmanEphemeral）**:基于椭圆曲线的密钥交换，具有更高的计算效率和安全性。

	后量子算法（如 Kyber）:在抗量子计算的 TLS 方案中逐渐被研究使用，以抵抗量子计算攻击。 2.对称加密算法 **功能**:使用共享密钥对数据进行加密，确保数据的机密性，防止数据在传输过程中被窃听。 **常见算法**: **AES（AdvancedEncryptionStandard）**:支持 128 位、192 位和 256 位密钥长度，是当前最常用的对称加密算法。 **ChaCha20**:提供高效且安全的加密，适合低功耗和移动设备。 **工作模式**: **GCM（Galois/CounterMode）**:结合了加密和消息认证，可以提供数据的保密性和完整性。 **Poly1305**:常与 ChaCha20 结合使用，作为其消息认证机制。 3.消息认证码（MAC）算法 **功能**:用于校验数据的完整性，确保数据在传输过程中没有被篡改。 **常见算法**: **HMAC-SHA256/SHA384**:基于 SHA-2 的哈希算法，生成数据完整性验证标签，用于防止数据被修改。 **Poly1305**:与 ChaCha20 搭配，为数据提供消息认证。 4.签名算法 **功能**:用于认证客户端和服务器的身份，防止身份伪造。 **常见算法**: **RSA**:常用于证书签名，但由于不具备前向安全性，在 TLS1.3 中逐渐被弱化。 **ECDSA**:基于椭圆曲线的签名算法，计算效率高，支持更短的密钥长度。

	EdDSA:新型签名算法，基于椭圆曲线，提供高效的签名和验证，且支持抗量子版本（Ed448）。 5.伪随机函数（PRF） **功能**:用于生成密钥材料，确保密钥的随机性和不可预测性。 **常见算法**: **HMAC-SHA256**:基于 SHA-2 的伪随机函数，用于生成会话密钥材料，以确保密钥交换的安全性。

3.2.迁移点分析与确认

通过对 TLS 的协议流程进行全面分析，确定 TLS 协议中公钥密码参与的环节，确定抗量子迁移点。

TLS 协议中公钥密码体系涵盖身份鉴别、消息完整性与密钥交换三大环节，本课题聚焦优化密钥交换算法，保留基于数字签名的身份鉴别和完整性验证机制。因私钥泄露仅威胁未来签名有效性，对现有通信影响有限，故将资源集中于提升密钥协商效率、强化前向安全性及降低计算开销，通过算法升级实现更安全高效的会话密钥建立。

TLS 密钥交换算法是实现安全通信的重要组成部分，其主要目的是在不安全的网络环境中安全地协商共享密钥。当前，TLS 协议主要依赖于经典的密钥交换协议，如 Diffie-Hellman（DH）和 EllipticCurveDiffie-Hellman（ECDH），这些协议基于数论的难题，例如离散对数问题，来确保密钥交换的安全性。因此，密钥交换环节的迁移点涉及公钥加密/封装、私钥解密/解封装等使用公钥的节点。在进行公钥加密/封装时，可以通过将加密/封装算法替换成抗量子算法，或者将抗量子加密/封装、解密/解封装过程进行密钥交换的抗量子迁移。

3.3.方案设计

在确认密码协议基本信息及密码应用位置后，本节对 TLS 协议抗量子迁移方案进行设计。

TLS 协议的抗量子迁移应以 TLS 1.3 握手流程为基础，对协议中涉及公钥密码算法的功能模块进行分析，识别需要进行抗量子改造的关键节点。TLS 1.3 协

议中，对称加密算法、哈希算法以及密钥派生机制目前尚未受到量子计算的直接威胁，通过适当增加安全参数即可继续使用，因此其并非本次迁移工作的重点。相比之下，协议中的密钥交换机制和数字签名机制均依赖于 RSA、ECDSA、ECDHE 等传统公钥密码算法，而这些算法在量子计算环境下均可能受到 Shor 算法攻击，因此需要重点开展抗量子迁移改造。

结合 TLS 1.3 握手流程分析可以发现，客户端在 ClientHello 消息中通过 KeyShare 扩展发送密钥交换参数，服务器在 ServerHello 消息中返回对应的密钥交换参数，双方基于协商结果生成共享密钥并进一步导出会话密钥。因此，密钥交换阶段是 TLS 协议抗量子迁移的重要节点，需要将现有基于 ECDHE 的密钥协商机制替换为混合密钥交换机制，以保证会话密钥在量子计算环境下仍具备安全性。同时，在服务器认证或双向认证场景下，TLS 协议通过 Certificate 消息传输数字证书，并通过 CertificateVerify 消息完成握手消息签名验证这里需替换成混合签名机制。因此，证书体系及数字签名机制同样构成 TLS 协议抗量子迁移的关键节点，需要将现有 RSA、ECDSA 等签名算法迁移为抗量子数字签名算法，并同步完成证书体系和签名算法协商机制的升级改造。

综上所述，TLS 协议抗量子迁移涉及的核心改造对象主要包括两类：一类是用于会话密钥协商的密钥交换算法，另一类是用于身份认证和握手消息完整性保护的数字签名算法。其中，密钥交换算法迁移主要对应 ClientHello 和 ServerHello 消息中的 KeyShare 扩展及相关密码套件协商机制；数字签名算法迁移主要对应 Certificate、CertificateVerify 以及 Signature Algorithms 扩展等认证相关模块。上述节点构成了 TLS 协议抗量子迁移的主要实施对象。

3.3.1. 基于签名的抗量子 TLS 协议设计方案

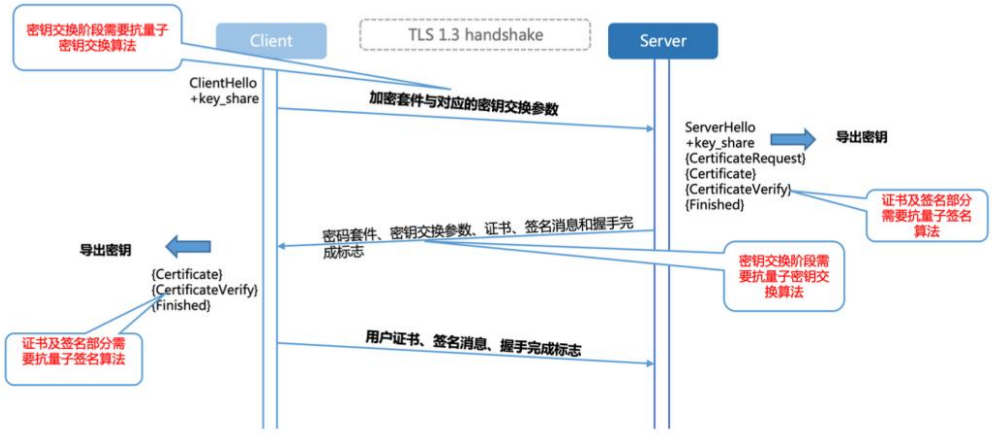


图 5 混合 TLS1.3 流程示意图

1. 算法选择

在抗量子算法选择上，采用 NIST 后量子标准化算法（Kyber 密钥封装、Dilithium 签名），但需评估其与 TLS 现有密码套件的兼容性。例如，Kyber 算法在 TLS1.3 中的集成需调整 KeyShare 扩展结构，可能导致部分旧版服务器因无法解析新字段而断开连接。同时，抗量子算法可能导致 TLS 握手失败（如 Chrome124 引入 X25519Kyber768 后部分服务器无法处理扩展的 ClientHello 消息），需确保服务器和中间件支持新算法或具备降级机制，因此本课题设计的混合 TLS 协议中除了支持 X25519Kyber768 密码交换套件还支持 X25519 的纯经典算法套件，同时，在经典算法选择上，提供不仅支持 X25519 算法还支持 SM2 算法。

2. 密钥交换机制调整

本方案中设计“传统算法+抗量子算法”双保险模式（如 TLS1.3 中同时使用 ECDHE 和 Kyber），设计可插拔式密码模块以支持算法快速替换。双方节点有一方不支持抗量子算法的时候，支持回退到经典密钥交换算法。

在 TLS1.3 协议中，ClientHello 是客户端发起的握手消息，用于协商支持的 TLS 版本、加密套件、扩展功能（如密钥交换算法）及生成随机数；ServerHello 则是服务器的响应，确认最终选择的协议参数、加密套件，返回证书链和服务器随机数。双方通过交换随机数生成主密钥，结合强制使用的 ECDHE 算法确保前向保密。为支持抗量子密码算法，需对握手阶段的密钥交换机制进行扩展：

（1） 客户端在 ClientHello 消息的 key_share 扩展字段中，新增混合经典/抗量子密码算法标识（例如 X25519Kyber768Draft00），表明其支持传统算法（如

ECDHE）与抗量子算法（如 Kyber）的协同使用。

（2） 服务端则在 ServerHello 的 key_share 扩展中，明确回传所选混合算法标识，并用客户端抗量子算法公钥加密共享密钥密文，实现抗量子密码与传统密码的平滑兼容，确保协议在未来量子计算威胁下的长期安全性。

3. 签名机制调整

本方案采用“传统签名算法+抗量子签名算法”双保险模式，在迁移过渡阶段同时支持 RSA 签名算法和 ML-DSA（Dilithium）抗量子签名算法。通过可插拔式密码模块设计，实现签名算法和证书体系的灵活切换与平滑升级。当通信双方存在不支持抗量子签名算法的情况时，系统可回退至传统签名算法完成身份认证，从而保证现网业务的兼容性和连续性。

在 TLS 1.3 协议中，服务器通过 Certificate 消息向客户端发送数字证书链，通过 CertificateVerify 消息对握手消息进行数字签名，以证明其拥有对应证书的私钥。客户端在双向认证场景下同样需要发送 Certificate 和 CertificateVerify 消息完成身份认证。当前 TLS 协议普遍采用 RSA 算法完成证书签名和握手消息签名，这些算法均依赖于大整数分解问题或离散对数问题，在量子计算环境下存在被破解的风险。因此，需要对 TLS 协议中的身份认证机制进行抗量子改造。

为支持抗量子数字签名算法，需对 TLS 握手阶段的身份认证机制进行扩展：

（1） 客户端在 ClientHello 消息的 signature_algorithms 扩展字段中，新增抗量子签名算法标识或混合签名算法标识，表明其支持相应的抗量子签名能力及验证能力。

（2） 服务端在 ServerHello 后续握手过程中，根据协商结果返回采用抗量子签名算法签发的数字证书，并在 CertificateVerify 消息中使用对应的抗量子私钥对握手消息摘要进行签名。客户端则利用证书中的公钥完成签名验证。对于混合签名模式，可同时验证经典签名和抗量子签名。

3.3.2. 流程优化的抗量子 TLS 协议设计方案

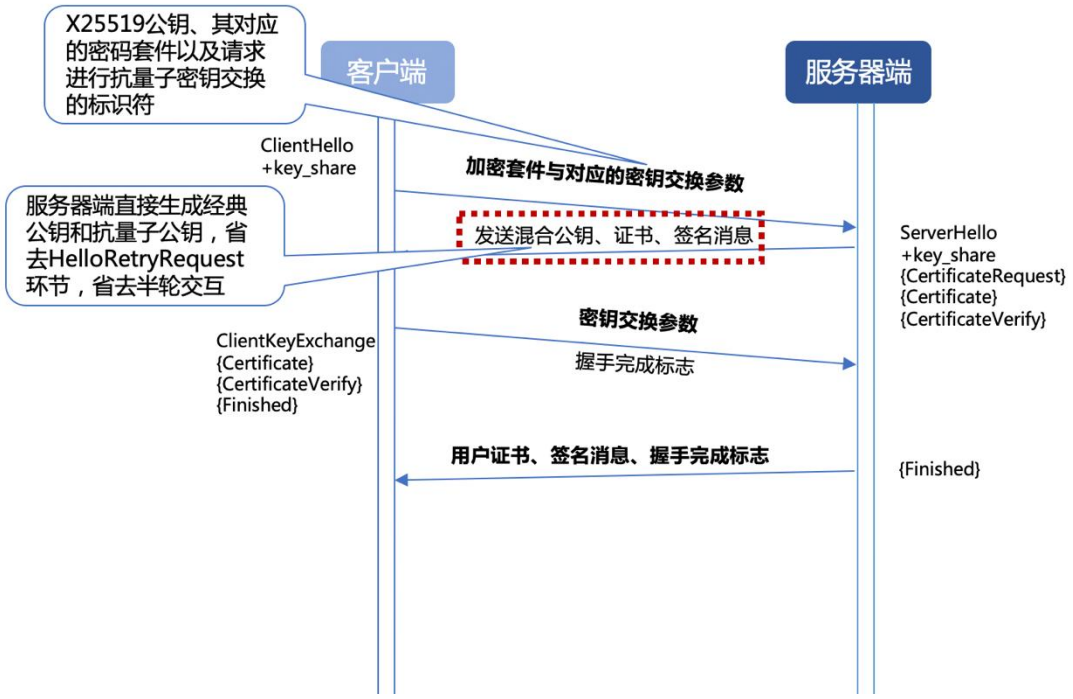


图 6 流程优化的抗量子 TLS 协议设计方案流程图

1. 方案设计概述

针对量子计算可破解 RSA、ECC 等传统公钥密码，现有谷歌 X25519Kyber 混合 TLS 方案依赖 HelloRetryRequest 重协商带来握手延迟高、中间盒兼容性差的缺陷，本文设计一套兼容传统椭圆曲线、国密 SM2、NIST 标准化后量子密码的混合 TLS 1.3 通信协议。方案采用分层可插拔密码模块，从算法选型、密钥交换机制、签名认证机制三个维度完成协议扩展改造，取消 HelloRetryRequest 重协商流程，仅通过单次握手往返完成混合密钥协商；同时兼容纯经典算法降级回退逻辑，兼顾存量中间件、老旧服务器设备适配性，在保障量子安全、国密合规的前提下降低握手时延，实现网络通信平滑后量子迁移。

2. 算法选择设计

（1） 抗量子密码算法选型

本方案完全采用 NIST 标准化后量子密码套件，密钥封装层选用 Kyber 系列格基 KEM 算法，身份签名层选用 Dilithium（ML-DSA）格基签名算法，两类算法均通过 NIST 后量子密码标准评审，具备标准化、高安全、工程易实现特性。

密钥交换算法：支持 X25519-Kyber512、X25519-Kyber768 两类混合密钥交换密码套件，对应标识 OQS_CODEPOINT_X25519_KYBER512 、X25519Kyber768Draft00；

身份签名算法：采用 Dilithium2、Dilithium3（ML-DSA）作为抗量子签名基础算法，用于证书签发、握手消息完整性签名。

（2） 经典密码与国密算法选型

为适配存量互联网设备与国产化业务场景，协议内置多套传统密钥交换、签名算法，作为降级兼容备选：

密钥交换算法：原生支持 X25519 椭圆曲线算法，同时兼容国产商用密码 SM2 密钥交换算法；

签名算法：兼容 RSA-PSS、ECDSA 传统签名算法，配套国密 SM2-SM3 签名套件，满足政务、金融国产化合规要求。

（3） 兼容性与降级机制设计

Kyber、Dilithium 公钥、签名数据长度远大于传统算法，直接放入 ClientHello 易造成报文超长分片，老旧中间件、负载均衡器、低版本服务器无法解析扩展字段，引发连接中断。针对该问题设计多级兼容降级策略：

多密码套件共存协商：客户端在 ClientHello 的 CipherSuites 列表同时推送混合抗量子套件、纯 X25519 经典套件、SM2 国密套件；

服务端自适应降级：若服务器 / 中间件不识别 Kyber、ML-DSA 扩展标识，自动放弃后量子算法，回退至 X25519 或 SM2 纯经典密钥交换流程；

轻量化首包约束：初始 ClientHello 仅携带轻量化传统算法公钥，不传输 Kyber 公钥，规避首包超长分片问题，从源头解决中间盒断连故障。

3. 密钥交换机制调整设计

（1） 双保险混合密钥交换架构

本方案采用「传统椭圆曲线密钥交换 + 后量子格基 KEM」双层并行密钥协商模式，实现双重安全防护：

传统安全层：基于 X25519/SM2 完成 ECDHE 密钥协商，保障无量子攻击场景下的通信安全、会话前向保密；

抗量子安全层：基于 Kyber KEM 完成密钥封装与解封装，抵御 Shor 算法带来的“现在截获、未来解密”量子攻击；

会话密钥融合：两端将传统椭圆曲线共享密钥、Kyber 解封装共享密钥联合哈希派生最终会话主密钥，任意一类算法不泄露即可保障会话安全。

（2） 可插拔密码模块设计

将密钥交换逻辑封装为独立可插拔模块，统一对外提供密钥生成、封装、解封装接口，支持算法快速替换与新增：

模块内置 X25519、SM2、Kyber512、Kyber768 四类密钥交换实现；

通信双方协商完成后动态加载对应算法实例，一方不支持后量子算法时自动切换至纯经典算法模块，无需重构 TLS 握手主流程。

（3） TLS 1.3 握手扩展改造（取消 HelloRetryRequest）

原生谷歌方案依靠 HelloRetryRequest 实现 Kyber 密钥传输，额外增加半轮握手往返，提升通信延迟。本方案省略重协商流程，仅通过一次往返完成混合密钥协商，对握手报文扩展改造如下。

ClientHello 报文扩展 ：

在 CipherSuites 写入混合抗量子套件、纯经典算法套件；

key_share 扩展仅填充 X25519/SM2 轻量化公钥，不携带 Kyber 公钥，控制首包长度；

新增 pq_algorithm_support 扩展字段，声明客户端支持 X25519-Kyber 混合密钥交换；

supported_groups 字段标注支持 X25519、SM2 传统椭圆曲线。

ServerHello 报文扩展：

服务端识别客户端混合算法标识后，无需返回 HelloRetryRequest，直接完成混合套件确认：

CipherSuite 选定 X25519-Kyber 混合套件；

key_share 扩展同步返回服务端 X25519 公钥与 Kyber 公钥；

pq_algorithm_support 回传确认，告知客户端服务端兼容后量子密钥交换。

ClientKeyExchange ： 报文承载 Kyber 密文

客户端收到 ServerHello 双公钥后，执行 Kyber 封装操作，将 Kyber 密文与本地传统公钥写入 ClientKeyExchange 发送至服务端，完成抗量子密钥材料传输。

（4） 完整单次往返密钥协商流程

客户端发送轻量化 ClientHello，声明混合算法支持，仅携带 X25519 公钥；

服务端响应 ServerHello，同步下发 X25519 与 Kyber 双公钥；

客户端发送 ClientKeyExchange，携带 Kyber 封装密文；

两端分别完成 ECDHE 协商、Kyber 解封装，融合生成会话密钥；

双向发送 Finished 消息校验握手完整性，握手完成进入加密业务通信。

4. 签名机制调整设计

（1） 双保险混合签名架构

迁移过渡期同时启用「传统签名算法 + 抗量子 ML-DSA 签名算法」并行验证机制，构建双层身份安全保障：

传统签名层：RSA-PSS、ECDSA、SM2-SM3，适配现有 X.509、国密 SM2 证书体系；

抗量子签名层：Dilithium2/3（ML-DSA），用于证书签发、CertificateVerify 握手签名，抵御量子伪造攻击；

混合验证逻辑：服务端证书同时携带传统签名与 Dilithium 抗量子签名，客户端可同时完成双重签名校验，任一校验通过即可完成身份认证。

（2） 可插拔签名模块设计

独立封装签名、验签模块，统一封装签名生成、签名验证接口，实现签名算法灵活切换；

内置 RSA、ECDSA、SM2、Dilithium 多套签名实现；

协商阶段识别两端签名能力，若客户端不支持 ML-DSA，自动切换至传统签名模块完成身份认证，保障业务连续性。

（3） TLS 握手身份认证扩展改造

TLS 1.3 依靠 Certificate、CertificateVerify 完成服务器与客户端身份校验，原有签名算法均存在量子脆弱性，本方案对签名协商流程扩展：

在 signature_algorithms 扩展字段新增 ML-DSA、SM2 签名算法标识，向服务端告知客户端支持抗量子、国密签名的验证能力。服务端证书基于 X.509 v3 扩展字段嵌入 Dilithium 抗量子公钥与签名，保留原有 RSA/ECC 基础字段；在 CertificateVerify 消息中，使用 Dilithium 私钥对握手消息摘要生成抗量子签名。

客户端解析证书中传统签名、Dilithium 抗量子签名，分别使用对应公钥完成验签；若设备不支持 ML-DSA，直接跳过抗量子验签，仅校验传统签名实现降级兼容。

5. 方案整体优势

低握手时延：取消 HelloRetryRequest 重协商流程，握手交互减少半轮往返，大幅降低弱网、高并发场景通信延迟；

全维度兼容：首包仅携带轻量化传统公钥，无超长报文分片问题；支持纯 X25519、SM2 降级回退，兼容老旧中间件、服务器；

双重量子安全防护：密钥交换采用 X25519+Kyber 双 KEM，身份认证采用传统签名 + Dilithium 混合签名，抵御量子截获攻击；

国产化适配：原生支持 SM2 国密密钥交换与 SM2-SM3 签名，适配政务、金融国产化 PKI 基础设施；

高可扩展性：密钥交换、签名均采用可插拔模块化设计，后续新增后量子算法、国密算法仅需补充模块实现，无需重构 TLS 握手核心流程。

6. 安全保障设计

算法隔离：传统密码、后量子密码独立运算、独立密钥派生，单一算法漏洞不会泄露完整会话密钥；

握手完整性保护：Finished 消息对全部握手报文哈希校验，中间人无法篡改密钥交换、签名协商内容；

前向安全：每次握手独立生成全新 X25519、Kyber、Dilithium 密钥材料，长期证书私钥泄露无法解密历史通信；

分级降级安全：降级至纯经典算法时保留 ECDHE 前向安全能力，最低限度保障通信基础安全。

3.3.3. 异构认证的抗量子 TLS 协议设计方案

本方案基于前文扩展 X.509 v3 抗量子证书架构，划分两类互补握手认证场景，分别适配签名证书发起握手、KEM 证书发起握手两种异构证书交互模式，全程复用 TLS 1.3 原生握手报文框架，仅依托扩展字段完成抗量子算法适配，实现异构证书间身份认证与密钥协商解耦，兼容 ML-DSA（Dilithium）、SPHINCS + 抗量子签名算法与 ML-KEM（Kyber）密钥封装算法。

场景一：抗量子签名证书实体发起握手

该场景下客户端作为握手发起方，持有嵌入 ML-DSA/SPHINCS + 公钥的抗量子签名证书，依托签名私钥完成主动身份确权；服务端持有搭载 ML-KEM 公钥的抗量子 KEM 证书，通过 KEM 封装 / 解封装完成密钥交换与间接身份认证，分工实现身份抗抵赖与轻量化密钥协商。

证书预置准备：

发起方在 X.509 v3 私有扩展字段写入抗量子签名公钥，证书签发算法绑定对应 ML-DSA、SPHINCS + 签名套件，用于主动身份声明；接收方在独立扩展字段嵌入 ML-KEM 公钥，依托 KEM 机制完成密钥协商与被动身份佐证，双方提前通过 PKI 链路交叉验证对方证书合法性。

握手初始化与证书交互：

客户端 ClientHello 报文携带抗量子签名、KEM 算法能力套件，声明异构认证支持；服务端通过 ServerHello 协商算法套件后，在 Certificate 消息返回自身 KEM 证书，对外公开 ML-KEM 公钥。

客户端认证与密钥封装：

客户端校验服务端 KEM 证书链合法后，使用服务端 ML-KEM 公钥对会话预共享密钥进行加密封装，经由 ClientKeyExchange 传输密文；同时使用自身抗量子签名私钥，对握手上文（密钥封装信息、会话随机数、协议版本）生成签名，通过 CertificateVerify 完成主动身份认证。

服务端核验与会话密钥生成：

服务端利用本地 KEM 私钥解封装预主密钥，解封装成功则间接验证客户端使用合法公钥，完成客户端间接身份认证；同时提取客户端证书内抗量子签名

公钥，校验握手签名合法性。双方基于解封装得到的预密钥，遵循 TLS 1.3 HKDF 规则派生最终会话密钥，后续业务流量由会话密钥加密传输。

场景核心逻辑：发起方以「签名 - 验签」实现主动可信身份认证，接收方以「KEM 解封装」完成被动间接认证，一体化完成身份不可抵赖性与轻量化密钥交换。

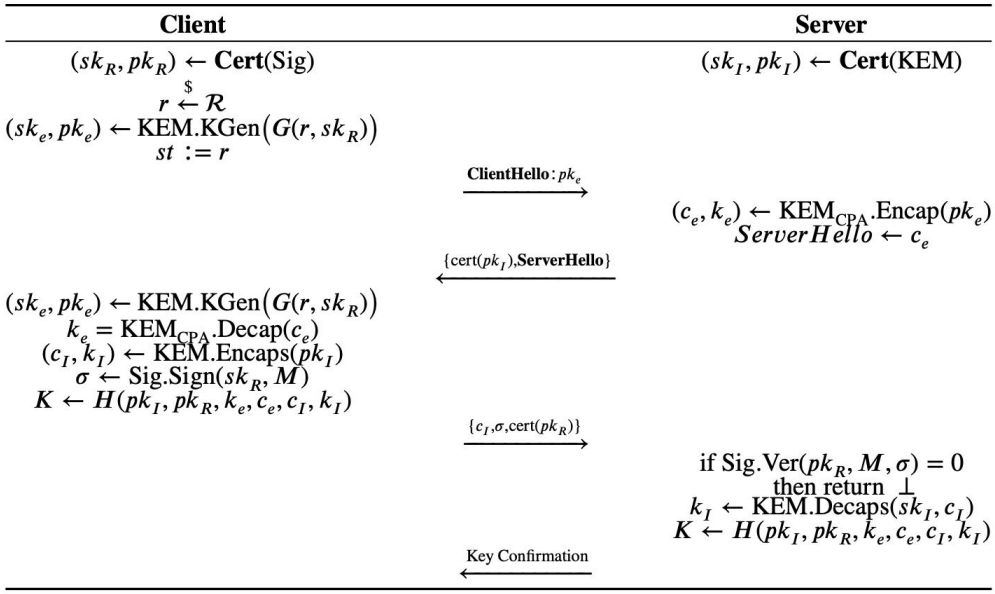


图 7 异构认证模式 TLS 协议（Sig-KEM）

场景二：抗量子 KEM 证书实体发起握手

本场景为场景一的互补模式，客户端持有抗量子 KEM 证书作为握手发起方，依靠 KEM 封装完成密钥交换与自身间接认证；服务端持有抗量子签名证书，以数字签名完成主动身份确权，适配服务端高可信身份背书、客户端轻量化部署场景。

证书预置准备：

发起方证书扩展字段嵌入 ML-KEM 公钥，负责密钥协商与间接自证；接收方证书搭载 ML-DSA/SPHINCS＋ 签名公钥，依靠抗量子签名完成服务端强身份认证，双方预校验证书合法性。

握手发起与服务端身份认证：

客户端 ClientHello 上报支持的抗量子算法套件，服务端响应 ServerHello 后，通过 Certificate 下发抗量子签名证书，同时使用签名私钥对握手上下文签名，经

由 CertificateVerify 发送；客户端使用服务端证书公钥验签，完成服务端主动身份合法性校验。

客户端密钥交换与间接认证：

客户端核验服务端签名证书无误后，上传自身 KEM 证书并公开 ML-KEM 公钥，服务端使用该公钥封装会话预主密钥，下发密钥密文，实现对客户端的间接身份认证。

客户端解密与会话建立：

客户端使用本地 KEM 私钥解封装预主密钥，解密成功即确认服务端身份合法；两端基于协商得到的密钥材料，通过标准 HKDF 函数派生会话密钥，完成 TLS 握手并加密传输业务数据。

场景核心逻辑：发起方依托 KEM 密钥封装实现间接身份认证，接收方依托抗量子签名完成主动身份认证，与场景一形成架构互补，适配不同终端算力、安全等级部署需求。

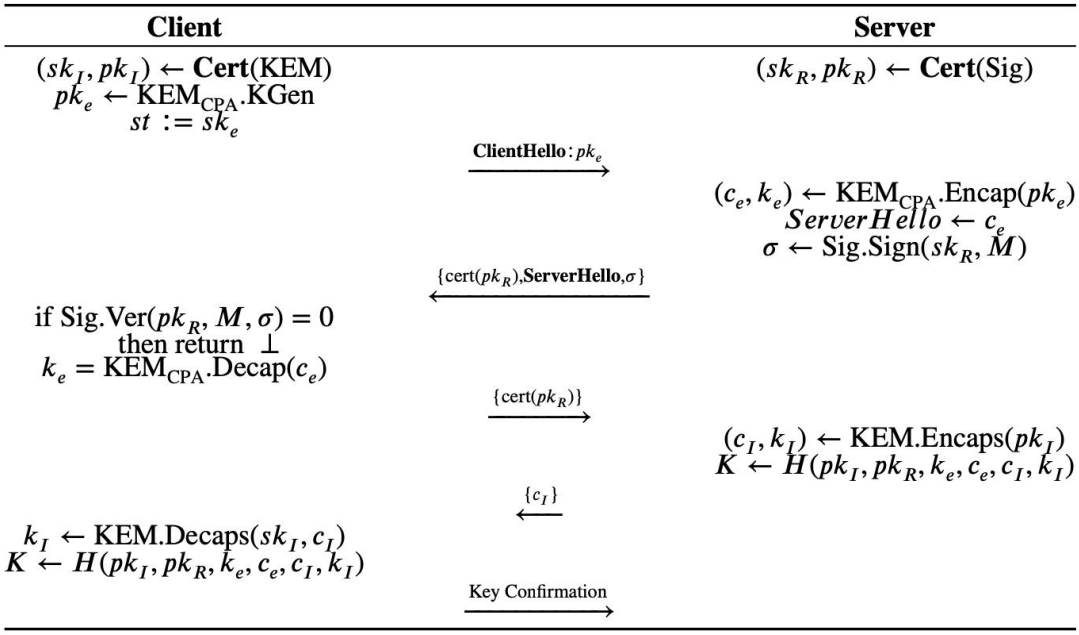


图 8 异构认证模式 TLS 协议（KEM-Sig）

1. 异构认证密钥派生规则

两类异构认证场景统一遵循 TLS 1.3 密钥派生规范，密钥输入材料融合抗量子密钥交换输出值与证书 KEM 封装预主密钥，将上述密钥材料作为 HKDF

密钥派生函数初始熵输入，结合客户端随机数、服务端随机数、协商密码套件标识，严格按照 TLS 1.3 标准 HKDF 提取、扩展步骤，分层派生握手阶段密钥、应用层业务加密密钥，保证密钥随机性、前向安全性与抗量子安全绑定能力。

2. 机制整体优势

协议兼容渐进部署：全程不修改 TLS 1.3 原生握手框架，仅基于 X.509 v3 扩展字段搭载抗量子公钥、签名材料，存量中间件、传统终端可忽略私有扩展字段平滑兼容，支持抗量子能力渐进式升级；

异构证书灵活适配：区分签名证书、KEM 证书两类角色分工，既可由客户端发起签名确权、服务端 KEM 密钥协商，也可反向部署，适配客户端轻量化物联网设备、高可信业务服务器等差异化算力场景；

双重安全兜底：签名链路保障身份不可抵赖性，KEM 密钥封装抵御量子计算截获破译风险，结合经典算法降级机制，同时满足国产化 SM2 兼容、后量子安全、现网业务连续性三重要求；

降低全域部署成本：无需全网统一部署单一类型抗量子证书，可根据节点安全等级、算力资源搭配签名型、KEM 型两类证书，拆分部署压力，降低 PKI 全域改造开销。

4. 分层模块详细功能设计

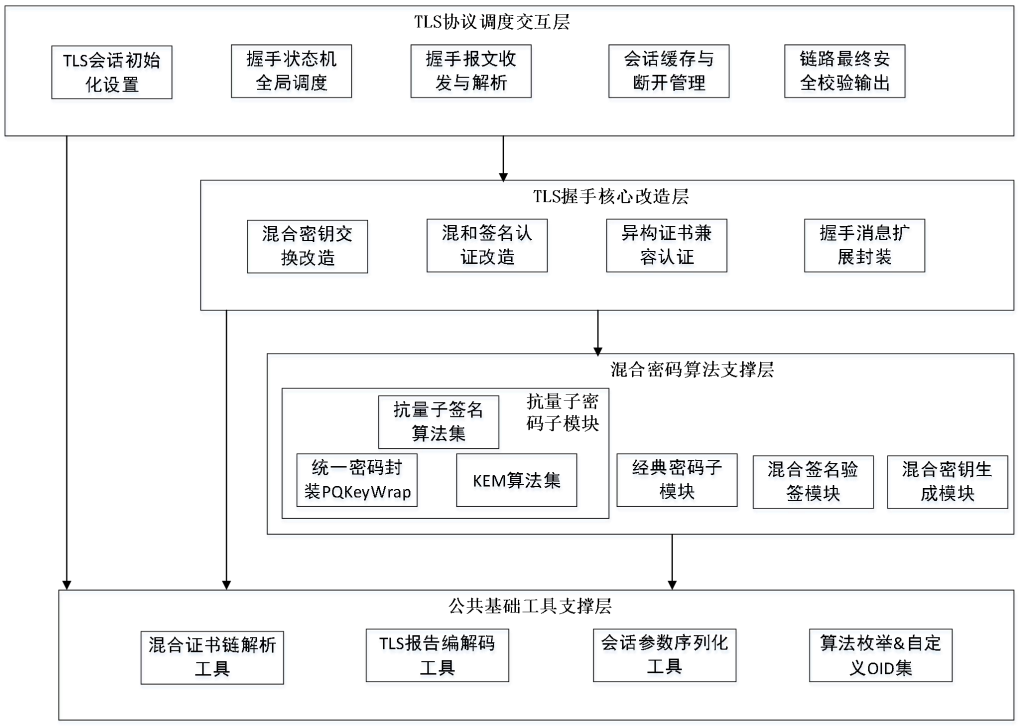


图 9 抗量子 TLS 协议整体框架

如图 9 所示，混合抗量子 TLS 协议采用与前文混合抗量子证书生成系统一致的四层模块化分层架构，自上而下依次为 TLS 协议调度交互层、TLS 握手核心改造层、混合密码算法支撑层与公共基础工具支撑层，各层级遵循上层单向调用下层的解耦设计原则，层级间依靠聚合式调用关系完成业务协同，无反向依赖与跨层级无序调用。顶层 TLS 协议调度交互层作为全协议统一入口，统筹会话初始化、握手状态机调度、报文收发与会话生命周期管理，向下驱动握手核心层完成协议改造逻辑；第二层 TLS 握手核心改造层为协议功能改造核心，分别实现混合密钥交换、混合签名认证、异构证书兼容适配、握手消息扩展封装四大核心改造能力，所有密码运算需求均下沉至下层密码支撑模块；第三层混合密码算法支撑层作为安全底座，划分经典密码子模块、后量子密码子模块、混合签名验签模块与混合密钥派生模块，依托统一密钥封装结构屏蔽后量子算法接口差异，向上提供兼容传统密码与抗量子密码的双重运算能力；最底层公共基础工具支撑层为全层级提供通用复用能力，涵盖混合证书解析、TLS 报文编解码、会话参数序列化、全局算法与扩展 OID 常量管理等基础工具，被上方三层所有模块共享

调用。整体架构视觉上采用分区底色区分层级、浅蓝色矩形标识独立功能模块，层级边界清晰、调用逻辑规整，在保障原生 TLS 1.3 协议框架完整性的前提下，实现安全能力模块化迭代、功能解耦与渐进式兼容部署的设计目标。

4.1. 公共基础工具支撑层

本层为全协议提供无状态通用基础能力，是协议运行的基础工具底座，所有上层握手、密码模块均可复用该层工具。模块内置全局算法枚举常量，统一管理 TLS 支持的经典算法与后量子算法标识，复用前文自定义 ASN.1 OID，用于识别混合证书内后量子公钥与签名字段；搭载混合证书链解析工具，可加载并解析自研根证书、中间证书、终端实体混合证书，完成证书基础合法性预解析；配套 TLS 标准报文编解码工具，兼容 TLS 1.3 原生 Handshake、Certificate、CertificateVerify、KeyShare 报文格式，实现扩展字段的无缝拼接与剥离；同时提供会话参数序列化、临时会话缓存读写工具，统一管理会话临时参数持久化与销毁，简化上层会话状态管理逻辑，消除重复工具代码。

4.2. 混合密码算法支撑层

该层为混合 TLS 协议提供经典 + 后量子双栈密码运算能力，分为经典密码子模块、后量子密码子模块、混合签名验签模块、混合密钥派生模块四部分，承接握手层所有密码运算请求。经典密码子模块提供 TLS 原生 ECDHE、ECDSA、RSA、HKDF 等标准算法，保障与传统未升级终端的互通兼容；后量子密码子模块封装 Dilithium、SPHINCS 抗量子签名算法与 Kyber 系列 KEM 密钥封装算法，通过统一 PQKeyWrap 结构体屏蔽不同后量子算法接口差异，向上提供标准化密钥生成、封装、解封装接口；混合签名验签模块专为 TLS 认证阶段设计，支持使用经典私钥与后量子私钥对握手上下文联合签名，校验端需同时通过经典公钥、后量子公钥双重校验才可判定身份合法；混合密钥派生模块将 ECDHE 经典协商密钥与 Kyber 封装解密后的后量子密钥拼接，送入 HKDF 函数生成最终会话主密钥，实现会话密钥的双重安全加固，抵御量子计算暴力破解。

4.3. TLS 握手核心改造层

该层是混合 TLS 协议的改造主体，基于 TLS 1.3 原生握手流程做增量扩展，不重构原有握手状态机，分为四大子模块实现分场景安全升级。

混合密钥交换改造模块针对 TLS KeyShare 报文进行扩展，在原有椭圆曲线共享值基础上新增 Kyber KEM 密钥封装参数，客户端同时发送 ECDHE 临时公钥与 Kyber 封装公钥，服务器返回 Kyber 解封装密文，完成经典密钥协商与后量子密钥协商并行执行；混合签名认证模块改造 CertificateVerify 握手消息，通信主体同时使用经典私钥、抗量子私钥对握手哈希上下文生成双重签名，接收端依托混合证书内两套公钥完成联合验签，强化身份认证抗量子安全性；异构证书兼容认证模块为本方案创新模块，专门适配一端持有 KEM 证书、另一端持有签名证书的异构通信场景，增加证书类型协商子流程，自动识别对方证书类型，完成 KEM 公钥密钥协商与签名公钥身份核验的跨类型适配，解决异构证书体系下握手失败问题；握手消息扩展封装模块负责在标准 TLS 握手消息 TLV 扩展域中搭载后量子算法标识、辅助参数，保证扩展内容不会被传统 TLS 解析器截断丢弃，实现渐进式兼容。

4.4. TLS 调度交互层

该层为协议最外层调度入口，负责 TLS 会话全生命周期管理与全局流程编排，对接底层握手模块与外部 TCP 网络数据流。会话初始化配置模块启动时加载全局算法策略、信任根混合证书、密钥套件优先级，配置优先后量子、兼容经典的降级策略；握手状态机全局调度严格遵循 TLS 1.3 握手流转时序，驱动客户端 Hello、服务器 Hello、证书传输、密钥协商、身份认证全阶段有序执行，异常状态下触发传统算法降级兜底；握手报文收发与解析模块封装底层 TCP 读写，完成网络字节流与 TLS 结构化报文相互转换，过滤非法恶意报文；会话缓存与断开管理模块维护成功协商后的会话主密钥、会话 ID，支持会话复用优化握手时延，连接关闭时安全销毁密钥上下文；最终链路安全校验在握手完成后整体校验证书链、签名合法性、会话密钥完整性，确认整条通信链路同时具备经典安全与抗量子安全能力，正式加密传输业务数据。

4.5. 模块整体交互执行层

客户端发起连接后，由顶层协议调度层初始化会话并发送 Client Hello，携带支持的后量子算法套件列表，握手核心层的混合密钥交换模块附加 Kyber KeyShare 参数；服务器接收报文后，调用混合密码层完成 Kyber 密钥解封装与 ECDHE 密钥协商，随后通过混合签名认证模块加载自身混合证书，在 Certificate 消息中上传证书链，通过 CertificateVerify 完成双重签名；若双方证书类型不一致，则由异构证书兼容模块启动跨类型认证适配，分别用 KEM 公钥完成密钥协商、签名公钥完成身份校验；密钥协商与身份认证全部完成后，混合密钥派生模块融合双栈密钥生成会话密钥，握手完成后调度层交付加密业务流量；全程所有证书解析、报文编解码、常量读取均由公共工具层统一支撑，密码运算统一调用下层混合密码支撑层，层级调用边界清晰，便于后续新增后量子算法、拓展更多异构证书适配规则。

5. TLS 协议抗量子迁移实现

本节给出 TLS 协议抗量子迁移的具体实现方案。针对 TLS 1.3 协议中密钥交换机制和身份认证机制存在的量子安全风险，对协议握手流程中的关键密码节点进行改造。具体内容包括 TLS 1.3 协议中 ClientHello、ServerHello、Certificate 和 CertificateVerify 等关键消息字段解析，抗量子密码算法相关协议扩展与标识定义，密钥交换与数字签名机制改造设计，预主密钥生成与会话密钥派生流程设计，以及兼容性、安全性和性能影响分析。通过引入“传统密码算法+抗量子密码算法”的混合模式，实现 TLS 协议从经典密码体系向后量子密码体系的平滑迁移，在保证现网兼容性的同时提升协议在量子计算环境下的长期安全性。

5.1. ClientHello 和 ServerHello 的字段解析

ClientHello 消息结构：

客户端发起的握手消息，用于协商协议参数、加密套件和扩展功能。主要包括以下字段：

ProtocolVersion：TLS1.3 的版本号。

Random：客户端生成的随机数，与服务器随机数共同生成主密钥。SessionID：TLS1.3 中通常为空（0x00），会话恢复改用 session_ticket 扩展。 CipherSuites：客户端支持的加密套件列表（按优先级排序），例如：TLS_AES_128_GCM_SHA256、TLS_CHACHA20_POLY1305_SHA2。

Extensions：

supported_versions：客户端实际支持的 TLS 版本（如 TLS1.3）。

key_share：客户端支持的密钥交换算法（如 ECDHE 曲线列表、SM2+Kyber512 混合命名组）。

signature_algorithms：客户端支持的签名算法列表，除传统 RSA-PSS、ECDSA 外，新增 sm2_dilithium2 混合签名标识，用于协商证书签发与握手身份认证使用的复合签名方案。

alpn：应用层协议协商（如 HTTP/2、HTTP/3）。

ServerHello 消息结构：

服务器响应客户端，用于确认协议参数、选定加密套件与签名算法、下发服务端证书链。主要包括以下字段：

ProtocolVersion：需与 supported_versions 扩展中的客户端支持版本一致。Random：服务器生成的随机数，与客户端随机数共同生成主密钥。 SessionID：通常为空（0x00），会话恢复依赖 session_ticket 扩展。 CipherSuite：服务器从客户端列表中选择的一个加密套件（如 TLS_AES_128_GCM_SHA256）。

CompressionMethod：固定为 0x00（无压缩）。

Extensions：

key_share：服务器选择的密钥交换算法及公钥参数，混合模式下携带服务端 SM2 公钥与 Kyber512 封装密文。

server_certificate：服务端证书采用混合签名 X.509 证书格式，由 CA 使用 SM2+Dilithium2 双重签名完成证书签发；协商降级场景可沿用传统单算法证书。

encrypted_extensions：加密后的扩展数据（如 ALPN 协议名）。 服务端依据客户端 signature_algorithms 扩展字段优先级，优先选用 sm2_dilithium2 混合签名方案，客户端不支持时自动回退至 ECDSA 或 RSA-PSS 传统签名。

5.2.协议扩展与标识定义

新增两类混合算法标识，分别对应密钥交换与签名两个模块： 在 TLS1.3 的 NamedGroup（命名组）中定义密钥交换混合标识符 sm2_kyber512_draft00；在 SignatureScheme 签名标识域新增混合签名标识符 sm2_dilithium2_draft00，分别用于密钥交换协商、X.509 证书签发与 CertificateVerify 握手签名协商。

1. 扩展密钥交换数据结构 改造 key_share 扩展字段的结构，使其支持同时携带经典算法公钥（SM2）和抗量子算法公钥（Kyber512）。具体为： ClientHello 的 key_share 中，客户端发送双公钥（SM2 公钥+Kyber512 公钥）及算法标识。ServerHello 的 key_share 中，服务端返回 SM2 公钥和 Kyber512 封装密文（封装后的共享秘密）。

2. 扩展签名相关数据结构 一是改造 signature_algorithms 与 signature_algorithms_cert 扩展，支持写入 sm2_dilithium2_draft00 混合签名标识；二是扩展 X.509 证书字段，在证书扩展域额外存储 Dilithium2 公钥与 Dilithium2 签名值，原有证书签名域保留 SM2 国密签名，形成双签证书；三是改造 CertificateVerify 载荷结构，签名字段由单一签名改为 SM2 签名拼接 Dilithium2 签名的复合数据格式。

5.3.ClientHello 和 ServerHello 消息改造

5.3.1. ClientHello 密钥交换相关改造

客户端密钥生成 客户端根据算法标识（如 sm2_kyber512_draft00）生成两对密钥： SM2 密钥对：用于经典密钥交换，生成共享秘密 K1。 Kyber512 密钥对：用于抗量子密钥封装，生成公钥。 消息字段填充 将算法标识写入 supported_groups 扩展，表明支持混合算法。在 key_share 扩展中，按固定顺序拼接 SM2 公钥和 Kyber512 公钥，附加算法标识后发送给服务端。

5.3.2. ClientHello 签名协商字段改造

客户端在 signature_algorithms 扩展列表首位填写 sm2_dilithium2_draft00，优先发起混合签名协商，列表后续依次填入 ECDSA、RSA-PSS 作为降级备选；若启用双向证书认证场景，客户端在 signature_algorithms_cert 同步声明自身证书支持 SM2+Dilithium2 混合签名。

5.3.3. ServerHello 密钥处理改造

服务端密钥处理，解析客户端的 key_share，提取 SM2 公钥和 Kyber512 公钥。 生成服务端 SM2 密钥对，与客户端 SM2 公钥计算共享秘密 K1。 生成临时共享秘密 K2，使用客户端的 Kyber512 公钥对其进行抗量子封装，生成密文。消息字段填充 在 key_share 扩展中返回服务端 SM2 公钥和 Kyber512 封装密文。算法标识与客户端协商一致，确保双方使用相同的混合算法流程。

5.3.4. ServerHello 签名协商处理

服务端遍历客户端 signature_algorithms 携带的签名方案列表，优先匹配 sm2_dilithium2_draft00，标记本次握手生效的混合签名算法；无匹配项时向后选取传统签名算法，记录至握手上下文，用于后续 CertificateVerify 消息签名生成。

5.4. 预主密钥生成流程

5.4.1. 密钥派生计算

客户端计算方式如下：

1、使用服务端 SM2 公钥与自身 SM2 私钥计算共享秘密 K1。 2、使用自身 Kyber512 私钥解封装服务端返回的密文，获得共享秘密 K2。 3、拼接 Z=K1||K2 作为预主密钥，输入后续密钥派生函数（HKDF）生成会话密钥。

服务端计算方式如下：

1、使用客户端 SM2 公钥与自身 SM2 私钥计算共享秘密 K1。 2、直接使用本地生成的 K2（无需解封装，因其已由客户端公钥封装）。 3、同样拼接 Z=K1||K2 生成预主密钥，确保双方密钥一致性。

5.4.2. 混合签名校验流程（CertificateVerify）

协商采用 sm2_dilithium2_draft00 方案时，服务端 CertificateVerify 消息采用复合签名： 1. 服务端对握手全量消息哈希摘要，分别使用自身证书 SM2 私钥生成签名 S1、Dilithium2 私钥生成后量子签名 S2，最终签名载荷格式为 S1||S2；2. 客户端先校验服务端 X.509 证书双签名：使用 CA 的 SM2 公钥核验证书 SM2 签名、CA 的 Dilithium2 公钥核验证书后量子签名，证书校验通过后，再使用服务端证书内 SM2、Dilithium2 公钥分别核验 S1 与 S2，双签名全部校验成功才认定服务端身份合法，任意一项验签失败直接终止握手流程。

5.5. 兼容性与安全性设计

5.5.1. 协商回退机制

1. 密钥交换降级：若服务端不支持混合算法，客户端通过 supported_groups 中的其他算法（如 X25519）进行传统密钥交换。客户端需优先列出混合算法标识，表明抗量子升级意愿。

2. 签名降级：服务端不识别 sm2_dilithium2_draft00 时，从 signature_algorithms 列表向后选用 ECDSA/RSA-PSS；老旧终端不支持混合签名标识时，自动忽略新增签名字段，沿用原有 TLS 单签握手逻辑。

5.5.2. 安全性增强

经典安全性：SM2 兼顾密钥交换与证书签名的国密合规身份认证，满足国内金融密码监管规范； 抗量子安全性：Kyber512 保障密钥交换共享秘密 K2 抵御量子分解攻击，Dilithium2 实现证书签名、握手身份签名抗量子伪造，即便 SM2 长期私钥未来被量子破解，存量历史签名无法被篡改伪造； 前向安全性：双方临时生成的 K1、K2 均为单次会话临时密钥，会话结束即销毁；Dilithium2 握手签名依托临时运算参数生成，不泄露证书长期签名私钥，进一步强化签名链路前向安全。

5.6. 协议改造影响分析

5.6.1. 消息长度分析

原有 key_share 开销：ClientHello 的 key_share 字段长度增加（SM2 公钥 64 字节+Kyber512 公钥 800 字节）。ServerHello 的 key_share 包含 SM2 公钥和 Kyber 密文（1088 字节）。 新增签名相关报文增量： 1. signature_algorithms 扩展仅新增 2 字节算法标识，长度开销可忽略； 2. CertificateVerify：传统 ECDSA 签名约 64 字节，混合模式为 SM2(64B)+Dilithium2(1704B)，单消息额外增加 1704B； 3. X.509 证书：附加 Dilithium2 公钥 1952B+Dilithium2 证书签名 1704B，单证书整体扩容约 3656B。

5.6.2. 网络性能分析

握手延迟增加：

1. 密钥交换字段额外~1952 字节在高延迟网络（如卫星链路）中可能增加 10-50ms 延迟；叠加 CertificateVerify 混合签名载荷，极端弱网场景握手延迟额外

上浮 5~20ms。

2. 低带宽场景（如 2G/3G 网络）报文体积变大，证书扩容大幅提升 TCP 分片概率，占用握手阶段带宽占比。 计算开销上升： 服务端解析超长 key_share 会带来原有 5%~15%CPU 负载；新增 SM2 签名+Dilithium2 格签名运算，服务端 CPU 负载在此基础额外上升 8%~22%；客户端双算法验签开销提升 15%~30%，可依托国密硬件加密卡、后量子算法专用硬件加速抵消算力损耗。

5.6.3. 兼容性分析

1. 中间设备兼容性： 部分防火墙、IDS、WAF 未适配超长 key_share、超长 CertificateVerify 与扩容证书，易触发异常报文拦截、误封；报文整体增大后更容易超出链路 MTU（VPN 隧道 1400 字节等限制），引发 TCP 分片异常、握手中断。

2. 客户端兼容性： 未升级老旧客户端无法识别混合签名标识与扩展字段，依靠 signature_algorithms、supported_groups 字段优先级自动降级至传统 TLS 握手，实现平滑兼容。

5.6.4. 安全性分析

1. 实现密钥交换+证书签名全链路抗量子防护，SM2 满足国内金融行业国密落地要求，Kyber、Dilithium 依托 NIST 后量子标准抵御 Shor 算法量子破解，高长度密钥与签名提升暴力破解门槛。

2. 潜在风险： 超长密钥、签名字段易在代码实现中出现缓冲区溢出漏洞；双签、双密钥交换逻辑提升代码复杂度，编码疏漏易造成验签/密钥派生安全漏洞，落地需严格遵循 RFC8446、GM/T 国密规范、NIST 后量子密码标准。 协议改造在密钥交换、证书签名双链路同步完成国密+后量子混合升级，在补齐量子安全短板的同时增大报文尺寸与算力消耗，工程落地可通过分场景渐进部署、报文压缩、硬件加速、全链路兼容性测试实现安全与性能平衡。

5.7. 协议模块实现方案（对应第 4 章模块设计图改）

5.7.1. 用户端

1. 用户应用接口模块

(1) 功能：为用户应用程序提供简洁易用的接口，负责接收用户的网络请求，并将其传递给传统 TLS 功能模块进行后续处理。同时，将从服务器端接收到的响应数据返回给用户应用程序。

(2) 实现方案：通过定义函数来封装接口逻辑。例如，定义 SendRequest 函数用于发送请求，ReceiveResponse 函数用于接收响应。在函数内部，构建合适的数据结构，并调用传统 TLS 功能模块的相应函数。

(3) 示例代码：

```
package main
import (
    "fmt"
)
// 假设传统 TLS 功能模块接口函数
func TraditionalTLSSendRequest(requestData string)
func TraditionalTLSReceiveResponse() string
// 用户应用接口模块函数
func SendRequest(userRequest string) {
    TraditionalTLSSendRequest(userRequest)
}
func ReceiveResponse() string {
    return TraditionalTLSReceiveResponse()
}
```

2. 传统 TLS 功能模块

(1) 功能：实现传统 TLS 协议的核心功能，包括 TLS 握手、数据加密与解密、连接管理等。在抗量子 TLS 迁移过程中，首先进行传统 TLS 握手，为后续的抗量子扩展握手奠定基础。

(2) 实现方案：利用 Go 语言的 crypto/tls 包来实现传统 TLS 功能。初始化 TLS 配置，加载证书和私钥等相关参数。在握手过程中，遵循 TLS 协议规范进行消息交互和验证。

(3) 示例代码：

```
package main
import (
    "crypto/tls"
    "fmt"
    "io/ioutil"
)
// 初始化 TLS 配置
func InitTLSConfig() (*tls.Config, error) {
    cert, err := tls.LoadX509KeyPair("client.crt", "client.key")
    if err!= nil {
        return nil, err
    }
    config := &tls.Config{
        Certificates: []tls.Certificate{cert},
        // 可根据需求添加更多配置
    }
    return config, nil
}
// 进行传统 TLS 握手
func TraditionalTLSHandshake(config *tls.Config, serverAddr string) (*tls.Conn, error) {
    conn, err := tls.Dial("tcp", serverAddr, config)
    if err!= nil {
```

```
            return nil, err
        }
        return conn, nil
}
// 发送数据
func TraditionalTLSSendData(conn *tls.Conn, data string) error {
        _, err := conn.Write([]byte(data))
        if err!= nil {
            return err
        }
        return nil
}
// 接收数据
func TraditionalTLSReceiveData(conn *tls.Conn) (string, error) {
        buffer := make([]byte, 1024)
        n, err := conn.Read(buffer)
        if err!= nil {
            return "", err
        }
        return string(buffer[:n]), nil
}
```

3. 抗量子 TLS 扩展模块

(1) 功能：负责实现抗量子 TLS 的扩展功能，包括在握手阶段协商抗量子加密算法、交换抗量子密钥、验证抗量子证书等。在数据传输阶段，使用抗量子加密算法对数据进行加密和解密。

(2) 实现方案：基于传统 TLS 功能模块，在握手过程中添加自定义的扩展消息，用于协商抗量子加密算法和交换密钥。利用抗量子加密库，如 pqcrypto 库中的相关算法（以 Kyber 为例），进行密钥生成、加密和解密操作。

(3) 示例代码：

```
package main
import (
    "encoding/hex"
    "fmt"
    "github.com/cloudflare/pqcrypto/kyber"
    "crypto/tls"
)
// 发送抗量子 TLS 扩展握手请求
func QuantumResistantExtensionRequest(conn *tls.Conn) error {
    publicKey, privateKey, err := kyber.GenerateKeyPair()
    if err!= nil {
        return err
    }
    // 假设将公钥通过自定义扩展消息发送
    // 这里简化为直接通过 TLS 连接发送
    _, err = conn.Write(publicKey[:])
    if err!= nil {
        return err
    }
    // 存储私钥用于后续解密
    // 实际应用中应妥善保管私钥
    // 这里仅为示例
```

```
	fmt.Printf("Generated private key: %s\n", hex.EncodeToString(privateKey[:]))
	return nil
}
// 接收抗量子 TLS 扩展握手响应
func QuantumResistantExtensionResponse(conn *tls.Conn) ([]byte, error) {
	receivedPublicKey := make([]byte, kyber.PublicKeySize)
	_, err := conn.Read(receivedPublicKey)
	if err!= nil {
		return nil, err
	}
	return receivedPublicKey, nil
}
// 使用抗量子加密发送数据
func QuantumResistantSendData(conn *tls.Conn, data string, publicKey []byte) error {
	encryptedData, err := kyber.Encrypt(publicKey, []byte(data))
	if err!= nil {
		return err
	}
	_, err = conn.Write(encryptedData)
	if err!= nil {
		return err
	}
	return nil
}
// 使用抗量子加密接收数据
```

```
func QuantumResistantReceiveData(conn *tls.Conn, privateKey []byte) (string, error) {
    encryptedData := make([]byte, 1024)
    n, err := conn.Read(encryptedData)
    if err!= nil {
        return "", err
    }
    decryptedData, err := kyber.Decrypt(privateKey, encryptedData[:n])
    if err!= nil {
        return "", err
    }
    return string(decryptedData), nil
}
```

4. 抗量子密钥管理模块

(1) 功能：生成、存储和管理抗量子密钥。与证书颁发机构的抗量子密钥管理与分发模块进行交互，获取最新的抗量子密钥，并将其提供给抗量子 TLS 扩展模块使用。

(2) 实现方案：使用抗量子加密库生成密钥对，将私钥安全存储在本地，例如使用文件加密存储。通过网络通信与证书颁发机构进行密钥更新和同步操作。

(3) 示例代码：

```
package main
import (
    "encoding/hex"
    "fmt"
    "github.com/cloudflare/pqcrypto/kyber"
    "io/ioutil"
    "net/http"
```

```
)
// 生成抗量子密钥对
func GenerateQuantumResistantKeyPair() ([]byte, []byte, error) {
	publicKey, privateKey, err := kyber.GenerateKeyPair()
	if err!= nil {
		return nil, nil, err
	}
	return publicKey[:], privateKey[:], nil
}
// 从证书颁发机构获取抗量子密钥（简化示例，实际需要更复杂的认证和通信）
func GetQuantumResistantKeyFromCA() ([]byte, error) {
	resp, err := http.Get("https://ca.example.com/get_key")
	if err!= nil {
		return nil, err
	}
	defer resp.Body.Close()
	keyData, err := ioutil.ReadAll(resp.Body)
	if err!= nil {
		return nil, err
	}
	return keyData, nil
}
```

5.7.2. 服务器端

1. 服务器应用接口模块

(1) 功能：与用户端的用户应用接口模块相对应，为服务器应用程序提供接口，接收来自用户端的请求数据，并将处理后的响应数据返回给用户端。

(2) 实现方案：通过定义函数来实现接口功能。例如，定义 ReceiveRequest 函数用于接收请求，SendResponse 函数用于发送响应。在函数内部，调用传统 TLS 功能模块和抗量子 TLS 扩展模块的函数来处理请求。

(3) 示例代码：

```
package main
import (
    "fmt"
)
// 假设传统 TLS 功能模块接口函数
func TraditionalTLSReceiveRequest() string
func TraditionalTLSSendResponse(responseData string)
// 服务器应用接口模块函数
func ReceiveRequest() string {
    return TraditionalTLSReceiveRequest()
}
func SendResponse(serverResponse string) {
    TraditionalTLSSendResponse(serverResponse)
}
```

2. 传统 TLS 功能模块

(1) 功能：与用户端的传统 TLS 功能模块类似，实现传统 TLS 协议的服务器端功能，包括 TLS 握手、数据加密与解密、连接管理等。在抗量子 TLS 迁移过程中，与用户端进行传统 TLS 握手。

(2) 实现方案：利用 Go 语言的 crypto/tls 包初始化 TLS 配置，加载服务器证书和私钥等参数。在握手过程中，验证用户端证书（如果需要），并进行密钥交换和会话建立。

(3) 示例代码：

```
package main
import (
	"crypto/tls"
	"fmt"
	"io/ioutil"
)
// 初始化 TLS 配置
func InitTLSConfig() (*tls.Config, error) {
	cert, err := tls.LoadX509KeyPair("server.crt", "server.key")
	if err!= nil {
		return nil, err
	}
	config := &tls.Config{
		Certificates: []tls.Certificate{cert},
		// 可根据需求添加更多配置
	}
	return config, nil
}
// 进行传统 TLS 握手
func TraditionalTLSHandshake(config *tls.Config, listener *tls.Listener) (*tls.Conn, error) {
	conn, err := listener.Accept()
	if err!= nil {
		return nil, err
	}
	tlsConn, ok := conn.(*tls.Conn)
	if!ok {
```

```
            return nil, fmt.Errorf("expected *tls.Conn, got %T", conn)
        }
        return tlsConn, nil
}
// 接收数据
func TraditionalTLSReceiveData(conn *tls.Conn) (string, error) {
        buffer := make([]byte, 1024)
        n, err := conn.Read(buffer)
        if err!= nil {
            return "", err
        }
        return string(buffer[:n]), nil
}
// 发送数据
func TraditionalTLSSendData(conn *tls.Conn, data string) error {
        _, err := conn.Write([]byte(data))
        if err!= nil {
            return err
        }
        return nil
}
```

3. 抗量子 TLS 扩展模块

(1) 功能：与用户端的抗量子 TLS 扩展模块协同工作，在握手阶段处理用户端发送的抗量子扩展请求，协商抗量子加密算法、交换抗量子密钥、验证抗量子证书等。在数据传输阶段，使用抗量子加密算法对数据进行加密和解密。

(2) 实现方案：在传统 TLS 握手完成后，接收用户端发送的抗量子扩展消息，解析并处理其中的抗量子公钥等信息。利用抗量子加密库进行密钥交换和数据加密操作。

(3) 示例代码：

```
package main
import (
	"encoding/hex"
	"fmt"
	"github.com/cloudflare/pqcrypto/kyber"
	"crypto/tls"
)
// 接收抗量子 TLS 扩展握手请求
func QuantumResistantExtensionRequest(conn *tls.Conn) ([]byte, error) {
	receivedPublicKey := make([]byte, kyber.PublicKeySize)
	_, err := conn.Read(receivedPublicKey)
	if err!= nil {
		return nil, err
	}
	return receivedPublicKey, nil
}
// 发送抗量子 TLS 扩展握手响应
func QuantumResistantExtensionResponse(conn *tls.Conn, publicKey []byte) error
{
	_, err := conn.Write(publicKey)
	if err!= nil {
		return err
	}
```

```
    return nil
}
// 使用抗量子加密接收数据
func QuantumResistantReceiveData(conn *tls.Conn, privateKey []byte) (string, error) {
    encryptedData := make([]byte, 1024)
    n, err := conn.Read(encryptedData)
    if err!= nil {
        return "", err
    }
    decryptedData, err := kyber.Decrypt(privateKey, encryptedData[:n])
    if err!= nil {
        return "", err
    }
    return string(decryptedData), nil
}
// 使用抗量子加密发送数据
func QuantumResistantSendData(conn *tls.Conn, data string, publicKey []byte) error {
    encryptedData, err := kyber.Encrypt(publicKey, []byte(data))
    if err!= nil {
        return err
    }
    _, err = conn.Write(encryptedData)
    if err!= nil {
        return err
    }
```

```
    return nil
}
```

4. 抗量子密钥管理模块

(1) 功能：与用户端的抗量子密钥管理模块类似，生成、存储和管理抗量子密钥。与证书颁发机构的抗量子密钥管理与分发模块进行交互，获取最新的抗量子密钥，并将其提供给抗量子 TLS 扩展模块使用。

(2) 实现方案：使用抗量子加密库生成密钥对，将私钥安全存储在本地。通过网络通信与证书颁发机构进行密钥更新和同步操作。

(3) 示例代码：

```
package main
import (
    "encoding/hex"
    "fmt"
    "github.com/cloudflare/pqcrypto/kyber"
    "io/ioutil"
    "net/http"
)
// 生成抗量子密钥对
func GenerateQuantumResistantKeyPair() ([]byte, []byte, error) {
    publicKey, privateKey, err := kyber.GenerateKeyPair()
    if err!= nil {
        return nil, nil, err
    }
    return publicKey[:], privateKey[:], nil
}
// 从证书颁发机构获取抗量子密钥（简化示例，实际需要更复杂的认证和通信）
```

```
func GetQuantumResistantKeyFromCA() ([]byte, error) {
	resp, err := http.Get("https://ca.example.com/get_key")
	if err!= nil {
		return nil, err
	}
	defer resp.Body.Close()
	keyData, err := ioutil.ReadAll(resp.Body)
	if err!= nil {
		return nil, err
	}
	return keyData, nil
}
```

6. 参考文献

[1] Shor P W. Polynomial-time algorithms for prime factorization and discrete logarithms on a quantum computer[J]. SIAM review, 1999, 41(2): 303-332.

[2] Rivest R L, Shamir A, Adleman L. A method for obtaining digital signatures and public-key cryptosystems[J]. Communications of the ACM, 1978, 21(2): 120-126.

[3] Koblitz N. Elliptic curve cryptosystems[J]. Mathematics of computation, 1987, 48(177): 203-209.

[4] Rescorla E. The transport layer security (TLS) protocol version 1.3[R]. 2018.

[5] Kent S, Seo K. Security architecture for the internet protocol[R]. 2005.

[6] Ylonen T, Lonvick C. The secure shell (SSH) protocol architecture[R]. 2006.

[7] Diffie, W., & Hellman, M. (1976). New Directions in Cryptography. IEEE Transactions on Information Theory, 22(6), 644-654.

[8] Neuman B C, Ts'o T. Kerberos: An authentication service for computer networks[J]. IEEE Communications magazine, 1994, 32(9): 33-38.

[9] Needham, R. M., & Schroeder, M. D. (1978). Using Encryption for Authentication in Large Networks of Computers. Communications of the ACM, 21(12), 993-999.

[10] Diffie, W., Van Oorschot, P. C., & Wiener, M. J. (1992). Authentication and Authenticated Key Exchanges. Designs, Codes and Cryptography, 2(2), 107-125.

[11] Kohl, J., & Neuman, B. C. (1993). The Kerberos Network Authentication Service (V5). Internet Engineering Task Force (IETF) RFC 1510.

[12] Harkins, D., & Carrel, D. (1998). The Internet Key Exchange (IKE). Internet Engineering Task Force (IETF) RFC 2409.

[13] Diffie, W., & Hellman, M. (1976). New Directions in Cryptography. IEEE Transactions on Information Theory, 22(6), 644-654.

[14] O'Gorman, L. (2003). Comparing Passwords, Tokens, and Biometrics for User Authentication. Proceedings of the IEEE, 91(12), 2021-2040.

[15] Hardt, D. (2012). The OAuth 2.0 Authorization Framework. Internet Engineering Task Force (IETF) RFC 6749.

[16] Daugman, J. (1993). High confidence visual recognition of persons by a test of statistical independence. IEEE Transactions on Pattern Analysis and Machine Intelligence, 15(11), 1148-1161

[17] Microsoft. (2019). Multi-Factor Authentication (MFA). Retrieved from Microsoft.

[18] Bellovin S M, Merritt M. Encrypted key exchange: Password-based protocols secure against dictionary attacks[C]//Proceedings 1992 IEEE Computer Society Symposium on Research in Security and Privacy. IEEE, 1992: 72-84.

[19] Bellare M， Namprempre C. Authenticated encryption: Relations among notions and analysis of the generic composition paradigm[C]//International Conference on the Theory and Application of Cryptology and Information Security. Berlin， Heidelberg: Springer Berlin Heidelberg， 2000: 531-545.

[20] Goldreich O， Lindell Y. Session-key generation using human passwords only[J]. Journal of Cryptology， 2006， 19: 241-340.

[21] O'Gorman, L. (2003). Comparing Passwords, Tokens, and Biometrics for User Authentication. Proceedings of the IEEE, 91(12), 2021-2040.

[22] Das, A., Bonneau, J., Caesar, M., Borisov, N., & Wang, X. (2014). The Tangled Web of Password Reuse. NDSS.

[23] Sakai, R., Ohgishi, K., & Kasahara, M. (2000). Cryptosystems based on pairing. In 2000 Symposium on Cryptography and Information Security (SCIS2000) (Vol. 2000, pp. 26-28).

[24] Jablon, D. P. (1996). Strong password-only authenticated key exchange. ACM SIGCOMM Computer Communication Review, 26(5), 5-26.

[25] Krawczyk, H. (2010). HMQV: A high-performance secure Diffie-Hellman protocol. In Annual International Cryptology Conference (pp. 546-566). Springer, Berlin, Heidelberg.

[26] Hoffstein, J., Pipher, J., & Silverman, J. H. (1998). NTRU: A ring-based public key cryptosystem. In International Algorithmic Number Theory Symposium (pp. 267-288). Springer, Berlin, Heidelberg.

[27] Avanzi, R., Bos, J., Ducas, L., Kiltz, E., Lepoint, T., Lyubashevsky, V., ... & Stehlé, D. (2020). CRYSTALS-Kyber. NIST Post-Quantum Cryptography Standardization.

[28] McEliece, R. J. (1978). A public-key cryptosystem based on algebraic. Coding Thv, 4244, 114-116.

[29] Niederreiter, H. (1986). Knapsack-type cryptosystems and algebraic coding

theory. Problems of Control and Information Theory, 15(2), 159-166.

[30] Ding, J., Schmidt, D., & Virtualized, V. (2005, January). Rainbow, a new multivariable polynomial signature scheme. In International Conference on Theory and Practice of Public-Key Cryptography (pp. 164-175). Springer, Berlin, Heidelberg.

[31] Patarin, J., Goubin, L., & Courtois, N. (1998, September). C*(--) Matriculation. In International Conference on the Theory and Application of Cryptology and Information Security (pp. 73-92). Springer, Berlin, Heidelberg.

[32] Bernstein, D. J., Hopwood, D., Hülsing, A., Lange, T., Niederhagen, R., Papachristodoulou, L., ... & Schwabe, P. (2015). SPHINCS: practical stateless hash-based signatures. In Annual International Conference on the Theory and Applications of Cryptographic Techniques (pp. 368-397). Springer, Berlin, Heidelberg.

[33] Hülsing, A., Rausch, L., & Buchmann, J. (2013). Optimal parameters for XMSS MT. In International Conference on Availability, Reliability, and Security (pp. 194-208). Springer, Berlin, Heidelberg.

[34] Gentry, C. (2009). Fully homomorphic encryption using ideal lattices. In Proceedings of the forty-first annual ACM symposium on Theory of computing (pp. 169-178).

[35] Brakerski, Z., Gentry, C., & Vaikuntanathan, V. (2012). (Leveled) fully homomorphic encryption without bootstrapping. ACM Transactions on Computation Theory (TOCT), 6(3), 1-36.

[36] Regev O. On lattices, learning with errors, random linear codes, and cryptography[J]. Journal of the ACM (JACM), 2009, 56(6): 1-40.

[37] Lyubashevsky V, Peikert C, Regev O. On ideal lattices and learning with errors over rings[C]//Advances in Cryptology–EUROCRYPT 2010: 29th Annual International Conference on the Theory and Applications of Cryptographic Techniques, French Riviera, May 30–June 3, 2010. Proceedings 29. Springer Berlin Heidelberg, 2010: 1-23.

[38] Alkim E, Ducas L, Pöppelmann T, et al. Post-quantum key {Exchange—A} new hope[C]//25th USENIX Security Symposium (USENIX Security 16). 2016: 327-343.

[39] Bos J, Ducas L, Kiltz E, et al. CRYSTALS-Kyber: a CCA-secure module-

lattice-based KEM[C]//2018 IEEE European Symposium on Security and Privacy (EuroS&P). IEEE, 2018: 353-367.

[40] Bos J W, Gourjon M O, Renes J, et al. Masking kyber: First-and higher-order implementations[J]. IACR Transactions on Cryptographic Hardware and Embedded Systems, 2021, 2021(4): 173-214.

[41] Ren P, Gu X. Practical post-quantum password-authenticated key exchange based-on module-lattice[C]//International Conference on Information Security and Cryptology. Cham: Springer International Publishing, 2021: 137-156.

[42] JARECKI S. Password Authenticated Key Exchange: Protocols and Security Models[J]. Asymmetric Cryptography: Primitives and Protocols, 2022: 213-255.

[43] Yang Y, Gu X, Wang B, et al. Efficient password-authenticated key exchange from RLWE based on asymmetric key consensus[C]//International Conference on Information Security and Cryptology. Cham: Springer International Publishing, 2019: 31-49.

[44] Karbasi A H, Atani R E, Atani S E. A New Ring-Based SPHF and PAKE Protocol on Ideal Lattices[J]. ISeCure, 2019, 11(1).

[45] Skrobot M. On Composability and Security of Game-based Password-Authenticated Key Exchange[J]. 2017.

[46] Ding J, Alsayigh S, Lancrenon J, et al. Provably secure password authenticated key exchange based on RLWE for the post-quantum world[C]//Cryptographers' Track at the RSA conference. Cham: Springer International Publishing, 2017: 183-204.

[47] Jintai Ding, Xiang Xie, and Xiaodong Lin. "A simple provably secure key exchange scheme based on the learning with errors problem". In: Cryptology ePrint Archive (2012).

[48] Joppe Bos, Craig Costello, Léo Ducas, Ilya Mironov, Michael Naehrig, Valeria Nikolaenko, Ananth Raghunathan, and Douglas Stebila. "Frodo: Take off the ring! practical, quantum-secure key exchange from LWE". In: Proceedings of the 2016 ACM SIGSAC conference on computer and communications security. 2016, pp. 1006–1018.

[49] Joppe W Bos, Craig Costello, Michael Naehrig, and Douglas Stebila. "Postquantum key exchange for the TLS protocol from the ring learning with errors problem". In: 2015 IEEE Symposium on Security and Privacy. IEEE. 2015, pp.

553–570.

[50] Erdem Alkim, Léo Ducas, Thomas Pöppelmann, and Peter Schwabe. “Postquantum Key {Exchange—A} New Hope”. In: 25th USENIX Security Symposium (USENIX Security 16). 2016, pp. 327–343.

[51] Tim Güneysu, Tobias Oder, Thomas Pöppelmann, and Peter Schwabe. “Software speed records for lattice-based signatures”. In: International Workshop on Post-Quantum Cryptography. Springer. 2013, pp. 67–82.

[52] Carlos Aguilar Melchor, Joris Barrier, Laurent Fousse, and Marc-Olivier Killijian. “XPIR: Private information retrieval for everyone”. In: Proceedings on Privacy Enhancing Technologies (2016), pp. 155–174.

[53] Jan-Pieter D’Anvers, Angshuman Karmakar, Sujoy Sinha Roy, and Frederik Vercauteren. “Saber: Module-LWR based key exchange, CPA-secure encryption and CCA-secure KEM”. In: International Conference on Cryptology in Africa. Springer. 2018, pp. 282–305.

[54] Joppe Bos, Léo Ducas, Eike Kiltz, Tancrède Lepoint, Vadim Lyubashevsky, John M Schanck, Peter Schwabe, Gregor Seiler, and Damien Stehlé. “CRYSTALSKyber: a CCA-secure module-lattice-based KEM”. In: 2018 IEEE European Symposium on Security and Privacy (EuroS&P). IEEE. 2018, pp. 353–367.

[55] Bin Wang, Xiaozhuo Gu, and Yingshan Yang. “Saber on ESP32”. In: International Conference on Applied Cryptography and Network Security. Springer. 2020, pp. 421–440.

[56] Joppe W Bos, Joost Renes, and Christine van Vredendaal. “Post-Quantum Cryptography with Contemporary Co-Processors: Beyond Kronecker, SchönhageStrassen & Nussbaumer”. In: Cryptology ePrint Archive (2020).

[57] Colin Boyd, Gareth T Davies, Bor de Kock, Kai Gellert, Tibor Jager, and Lise Millerjord. “Symmetric key exchange with full forward security and robust synchronization”. In: Advances in Cryptology–ASIACRYPT 2021: 27th International Conference on the Theory and Application of Cryptology and Information Security, Singapore, December 6–10, 2021, Proceedings, Part IV. Springer. 2021, pp. 681–710.

[58] Zongqu Zhao, Shaoti Ma, and Panke Qin. “Password authentication key exchange based on key consensus for IoT security”. In: Cluster Computing 26.1

(2023), pp. 1–12.

[59] Bindel N, Hale B. A note on hybrid signature schemes[J]. Cryptology ePrint Archive, 2023.

[60] Schanck J M, Whyte W, Zhang Z. Quantum-safe hybrid (QSH) ciphersuite for Transport Layer Security (TLS) version 1.2[J]. IETF, Internet-Draft draft-whyte-qsh-tls, 2016.

[61] Nagy N, Alnemer S, Alshuhail L M, et al. Module-lattice-based key-encapsulation mechanism performance measurements[J]. Sci, 2025, 7(3): 91.

[62] Alagic G, Bros M, Ciadoux P, et al. Status report on the first round of the additional digital signature schemes for the nist post-quantum cryptography standardization process[J]. 2024.

[63] Moody D, Dang Q. Additional SLH-DSA Parameter Sets for Limited Signature Use Cases[R]. National Institute of Standards and Technology, 2026.

[64] Bos J, Ducas L, Kiltz E, et al. CRYSTALS-Kyber: a CCA-secure module-lattice-based KEM[C]//2018 IEEE European symposium on security and privacy (EuroS&P). IEEE, 2018: 353-367.

[65] Aragon N, Barreto P, Bettaieb S, et al. BIKE: bit flipping key encapsulation[J]. 2022.

[66] Galbraith S D, Petit C, Shani B, et al. On the security of supersingular isogeny cryptosystems[C]//International conference on the theory and application of cryptology and information security. Berlin, Heidelberg: Springer Berlin Heidelberg, 2016: 63-91.

[67] Stebila D, Mosca M. Post-quantum key exchange for the internet and the open quantum safe project[C]//International Conference on Selected Areas in Cryptography. Cham: Springer International Publishing, 2016: 14-37.

[68] Schanck J M, Whyte W, Zhang Z. Quantum-Safe Hybrid (QSH) Ciphersuite for Transport Layer Security (TLS) version 1.2[R]. IETF Internet-Draft draft-whyte-qsh-tls12-01, Jan. 2016.

[69] Campagna M, Crockett E. BIKE and SIKE Hybrid Key Exchange Cipher Suites for Transport Layer Security (TLS)[R]. IETF Internet-Draft draft-campagna-tls-bike-sike-hybrid-00, Mar. 2019.

[70] BOS J W, COSTELLO C, NAEHRIG M, et al. Post-quantum key exchange for the TLS protocol from the ring learning with errors problem[C]//2015 IEEE

Symposium on Security and Privacy. San Jose: IEEE Computer Society, 2015: 553-570. DOI:10.1109/SP.2015.40.

[71] Bos J W, Costello C, Ducas L, et al. Frodo: Take off the Ring! Practical, Quantum-Secure Key Exchange from LWE[C]//2016 ACM SIGSAC Conference on Computer and Communications Security. ACM, 2016: 1006-1018.

[72] Stebila D, Mosca M. Post-quantum key exchange for the internet and the open quantum safe project[C]//Selected Areas in Cryptography–SAC 2016. Springer, 2017: 1-24.

[73] DING J T, GAO X W, LI L, et al. Practical Randomized RLWE-Based Key Exchange Against Signal Leakage Attack[J]. International Journal of Network Security, 2018, 20(6): 931-938. DOI:10.6633/IJNS.20181120(6).15.

[74] Kwiatkowski K, Sullivan N, Langley A, et al. Measuring TLS key exchange with post-quantum KEM[C]//2nd NIST Post-Quantum Cryptography Standardization Conference, 2019.

[75] Zhong Y T, Huang W Q, Yang Y, et al. Research on Post-Quantum Resistant Transport Layer Security Protocol[J]. Network Security Technology & Application, 2020(12):25-28.

[76] Zhang F, Pan T Y, Zhao Y L. Post-Quantum Migration Scheme, Implementation and Performance Evaluation for TLS 1.3[J]. Journal of Cryptologic Research, 2022, 9(1): 143-163.

国产商用密码与抗量子密码混合的TLS协议源码库
测试大纲

编　　写 _____

校　　对 _____

批　　准 _____

文件状态表

文档名称：国产商用密码与抗量子密码混合的 TLS 协议源码库测试大纲			
文档版本	修订日期	责任人	修订说明
V1.0	2026.6.9	刘凯博	编写测试用例

目 录

1 测试任务 1
1.1 任务概述 1
1.2 研究内容 1
1.3 考核指标 1
1.4 验收测试准则 2
2 被测对象说明 2
2.1 功能概述 2
2.2 测试范围 3
3 依据和引用文件 3
3.1 测试依据 3
4 测试技术及方法 4
4.1 功能测试方法 4
4.2 性能测试方法 4
4.3 兼容性测试方法 4
5 测试环境 4
5.1 硬件环境 4
5.2 软件环境 5
6 考核指标 5
7 问题处理 5
8 测试进度及人员安排 5
9 附录 A 软件问题报告单 7
10 附录 B 验收测试用例集 8
1 混合抗量子 TLS 协议功能测试 9
1.1 密钥交换功能测试验证 9
1.2 协议握手消息格式测试验证 11
1.3 密钥交换协商策略验证 12
1.4 异常交互处理测试验证 12
1.5 证书校验测试验证 13
1.6 加密通信测试验证 14
2 混合抗量子 TLS 协议性能测试 14
2.1 TLS 单连接性能测试 14
2.2 TLS 多连接性能测试 16
2.3 服务器资源消耗测试 16
3 混合抗量子 TLS 协议兼容性测试 17
3.1 协商结果事件记录验证测试 17
3.2 密钥材料导出验证测试 17
3.3 密钥交换回退验证测试 18

3.4 跨平台编译与运行验证测试.....19
4 测试结果总结.....21

1 测试任务

1.1 任务概述

中国科学院信息工程研究所承担《银行业及其关键基础设施信息系统的抗量子密码迁移技术研究任务书》中课题三"银行业典型信息系统抗量子密码安全协议迁移技术"的研究工作，研制了抗量子密码混合 TLS 协议源码库。测试组负责对该课题成果进行验收测试，验证其是否满足考核指标要求。

1.2 研究内容

本课题针对传统 TLS 协议存在的量子计算安全风险，研究国产商用密码与抗量子密码混合的 TLS 协议。研究工作遵循 RFC 8446 标准。在混合密码体系设计方面，将国密 SM2、X25519 等经典签名与密钥交换算法，与 Kyber 抗量子算法进行组合，定义 SM2Kyber512Draft00、X25519Kyber768Draft00 等混合加密交换组，完成协议消息结构和字段规则的设计与编码。

在握手协议方面，规范 ClientHello、ServerHello 等交互消息格式，实现协议版本、加密交换组、混合公钥等参数的协商。设计证书有效性校验模块，识别证书过期、签名错误等异常证书，阻断不安全握手。针对非法协议版本、错误密钥交换组等异常场景，实现拒绝连接、Hello Retry Request 等异常响应。在传输层面，完成数据加解密逻辑开发，支持文本、二进制等数据类型的安全传输。

在性能与工程适配方面，测试协议在单连接和高并发场景下的连接建立耗时，监测服务器 CPU、内存等资源消耗。实现协议向下兼容 TLS 1.2，完成 Windows、Linux 操作系统下的编译运行适配。

通过上述工作，建成混合 TLS 协议源码库，支持经典密码算法的兼容性与抗量子攻击能力，满足银行业及关键基础设施信息系统向抗量子密码体系迁移的需求。

1.3 考核指标

根据课题三考核指标 1.2“银行信息系统抗量子/混合密码安全协议库”，指

标完成状态要求：国产商用密码与抗量子密码混合的 **TLS**、IPSec、SSH 协议源码库，支持 2 类抗量子密码算法，可达到 128、256 比特的量子安全等级。

1.4 验收测试准则

依据《银行业及其关键基础设施信息系统的抗量子密码迁移技术研究任务书》，测试组针对《银行业及其关键基础设施信息系统的抗量子密码迁移技术研究任务书》中课题三“银行业抗量子密码安全协议库”的研究内容中考核指标进行测试。验收准则如下：

- 软件系统符合《银行业及其关键基础设施信息系统的抗量子密码迁移技术研究任务书》中课题三“银行业抗量子密码安全协议库”规定的全部功能要求；
- 验收测试过程中发现的问题全部闭环。

2 被测对象说明

本次测试的被测对象为国产商用密码与抗量子密码混合的 TLS 协议源码库。该源码库实现的 TLS 协议遵循 RFC 8446 标准，在 TLS 协议基础上将 SM2、X25519 等经典密钥交换算法与 Kyber 抗量子算法进行组合，支持 SM2Kyber512Draft00、X25519Kyber512Draft00、X25519Kyber768Draft00 等混合密钥交换组，并保留 X25519 传统密钥交换组用于兼容性回退验证。协议通过 ClientHello 和 ServerHello 消息完成 TLS 版本、supported_groups、key_share 等参数协商，支持非法协议版本拒绝、密钥交换参数不匹配场景下的 HelloRetryRequest 处理，以及无法协商时的连接拒绝。握手完成后，客户端和服务端通过 TLS ExportKeyingMaterial 导出会话密钥，并基于 AES-GCM 对应用层请求和响应数据进行加解密。协议源码支持在 Windows 和 Linux 操作系统下编译运行。

2.1 功能概述

主要功能列表如下：

序号	功能	功能子项	功能描述
1	混合密钥交换	SM2Kyber512Draft00； X25519Kyber512Draft00； X25519Kyber768Draft00	客户端和服务端可配置上述混合密钥交换组，并在 TLS 1.3 握手中协商共同支持的算法。

序号	功能	功能子项	功能描述
2	传统算法回退	X25519	当通信双方同时配置 X25519 时，可使用该传统密钥交换组建立 TLS 1.3 连接，用于兼容性验证。
3	协商结果记录	CFEventTLS13NegotiatedKEX	客户端通过事件处理函数获取并输出实际协商的密钥交换组。
4	证书有效期检查	已过期证书；未生效证书	客户端检查服务端证书的 NotBefore 和 NotAfter；检查失败时终止请求。
5	应用数据加密通信	密钥材料导出；AES-GCM 加解密	握手成功后导出 32 字节密钥材料，对 POST 请求及响应数据进行 AES-GCM 加解密。

2.2 测试范围

功能测试指根据《银行业典型信息系统抗量子密码安全协议迁移技术》课题任务书，对 TLS 协议源码库的考核指标覆盖功能进行验证性测试，重点验证 SM2Kyber512Draft00、X25519Kyber512Draft00、X25519Kyber768Draft00 等混合密钥交换组的协商能力，以及 ClientHello/ServerHello 消息格式、异常交互处理、证书有效期校验和加密通信功能。

性能测试指测试 TLS 协议在选取不同混合密钥交换组时，客户端与服务端建立 TLS 1.3 连接的响应时间，并记录单连接、多连接或并发场景下的连接成功率、平均耗时和资源消耗情况。兼容性测试指验证协议在保留传统 X25519 密钥交换组时的兼容性回退能力。该项用于验证迁移过程中的传统算法兼容能力，不计入抗量子混合密钥交换算法覆盖。

工程适配测试指验证 TLS 协议源码库在 Windows 和 Linux 操作系统目标环境下的编译能力，并验证 Linux 环境下服务端和客户端可完成 TLS 连接建立及加密通信。

3 依据和引用文件

3.1 测试依据

本验收测试大纲所依据和引用的文件见表 3-1。

表 3-1 依据和引用文件

分类	标识	文档名称	版本	发布日期	颁布/来源单位

分类	标识	文档名称	版本	发布日期	颁布/来源单位
技术标准	RFC 8446	The Transport Layer Security (TLS) Protocol Version 1.3	—	2018.08	Internet Engineering Task Force

4 测试技术及方法

4.1 功能测试方法

采用源码配置检查、程序运行输出检查、日志检查和协议交互结果比对等方式，验证功能正确性。测试过程中通过客户端和服务端日志确认 TLS 连接是否建立成功，通过协商结果事件记录确认实际使用的密钥交换组，通过请求和响应内容确认加密通信是否正确完成。

4.2 性能测试方法

采用重复执行客户端与服务端连接测试的方式，采集不同混合密钥交换组下 TLS 连接建立耗时。测试记录包括测试次数、成功次数、失败次数、平均耗时、最大耗时、最小耗时等数据。对于并发或多连接场景，记录连接成功率、平均响应时间以及服务端 CPU、内存等资源消耗情况，用于评估协议源码库在不同密钥交换组下的运行开销。

4.3 兼容性测试方法

在客户端和服务端同时保留 X25519 传统密钥交换组的条件下，验证当混合密钥交换组不可用或未被共同支持时，协议是否能够协商传统 X25519 密钥交换并完成 TLS 连接建立。该测试用于验证迁移过程中的兼容性回退能力，测试结果应单独记录，不计入抗量子混合密钥交换算法覆盖。

5 测试环境

5.1 硬件环境

名称	型号	备注
笔记本电脑	Dell Precision 3530	处理器：Intel Core i7-8750H*8

		内存：16GB 操作系统：Ubuntu 24.04.4 LTS，64 位

5.2 软件环境

名称	软件版本	备注
Go	go1.22.12 linux/amd64	TLS 协议源码库编译与运行

6 考核指标

依据验收测试要求进行了全面的需求分析，分解的测试项能够覆盖课题研究相关任务书要求，其追溯关系见下表。

考核指标	完成时指标值/状态	测试设计
指标 1.2 银行信息系统抗量子/混合密码安全协议库	国产商用密码与抗量子密码混合的 **TLS**、IPSec、SSH 协议源码库，支持 2 类抗量子密码算法，可达到 128、256 比特的量子安全等级	测试 TLS 协议源码库支持 SM2Kyber512Draft00、X25519Kyber512Draft00 等混合密钥交换组，覆盖 TLS 握手消息格式验证、混合密钥交换协商、异常交互处理、证书有效期校验、加密通信、协商结果记录、兼容性回退和跨平台编译运行等测试项。其中，Kyber-512 对应 128 比特量子安全等级，Kyber-768 对应 256 比特量子安全等级。

7 问题处理

对测试过程中发现的问题进行分析汇总，提交研制方进行问题确认，研制方应对问题进行解释，并提出处理意见（同意修改或不修改），对不修改问题说明理由。研制方根据处理意见对软件进行更改，测试方对更改后的软件完成更改确认和回归测试。

8 测试进度及人员安排

测试进度安排见下表。

序号	测试活动名称	开始日期	结束日期	负责人
1	测试需求分析与测试用例设计	2026.4.15	2026.5.15	刘凯博
2	测试大纲撰写	2026.5.16	2026.6.28	刘凯博
3	测试执行	2026.7.10	2026.7.20	刘凯博
4	回归测试	2026.7.20	2026.8.1	张航

序号	测试活动名称	开始日期	结束日期	负责人
5	测试总结	2026.8.1	2026.8.5	张航

9　附录 A 软件问题报告单

测试问题报告单

测试日期	测试人员	测试问题数量
测试用例标识	测试用例名称	测试问题描述

10 附录 B 验收测试用例集

测试用例集

1 混合抗量子 TLS 协议功能测试

1.1 密钥交换功能测试验证

测试用例名称	SM2Kyber512Draft00 密钥交换功能测试	标识	TLS-1-1
测试用例描述	验证 TLS 协议源码库是否支持 SM2Kyber512Draft00 混合密钥交换组。客户端和服务端均配置支持 SM2Kyber512Draft00 后，发起 TLS 握手连接，检查双方是否能够完成 ClientHello/ServerHello 参数协商，建立 TLS 连接，并通过协商结果日志确认实际使用的密钥交换组为 SM2Kyber512Draft00。		
前提和约束	配置支持 SM2Kyber512Draft00 的 TLS 客户端和服务端程序。		

测试步骤

序号	输入及操作	期望结果与评估标准	实测结果
1.	启动支持 SM2Kyber512Draft00 的 TLS 服务端程序。	服务端启动成功，监听测试端口。	
2.	客户端配置 SM2Kyber512Draft00 并发起 TLS 1.3 连接。	客户端和服务端完成 TLS 握手。	
3.	查看客户端输出日志或协商结果记录。	实际协商的密钥交换组为 SM2Kyber512Draft00。	

测试阶段	☐自测 ☐第三方测试		
通过情况		测试人员	
测试时间		测试地点	

测试用例名称	X25519Kyber512Draft00 密钥交换功能测试	标识	TLS-1-2
测试用例描述	客户端和服务端均配置支持 X25519Kyber512Draft00 混合密钥交换组，发起 TLS 1.3 握手连接，验证 TLS 协议源码库能够使用 X25519 与 Kyber-512 组合完成混合密钥交换协商。该用例用于检查 128 比特量子安全等级对应的 Kyber-512 混合密钥交换能力，确认客户端发送的 ClientHello 中包含该密钥交换组，服务端能够选择该组并返回正确的 ServerHello，最终双方成功建立 TLS 连接。		
前提和约束	配置支持 X25519Kyber512Draft00 的 TLS 客户端和服务端程序。		

测试步骤

序号	输入及操作	期望结果与评估标准	实测结果
1.	启动支持 X25519Kyber512Draft00 的 TLS 服务端程序。	服务端启动成功，监听测试端口。	
2.	客户端配置 X25519Kyber512Draft00 并发起 TLS 1.3 连接。	客户端和服务端完成 TLS 握手。	
3.	查看客户端输出日志或协商结果记录。	实际协商的密钥交换组为 X25519Kyber512Draft00。	

测试阶段	☐自测 ☐第三方测试		
通过情况		**测试人员**	
测试时间		**测试地点**	

测试用例名称	X25519Kyber768Draft00 密钥交换功能测试	**标识**	TLS-1-3
测试用例描述	客户端和服务端均配置支持 X25519Kyber768Draft00 混合密钥交换组，发起 TLS 1.3 握手连接，验证 TLS 协议源码库能够使用 X25519 与 Kyber-768 组合完成混合密钥交换协商。该用例用于检查 256 比特量子安全等级对应的 Kyber-768 混合密钥交换能力，确认客户端和服务端能够完成参数协商并成功建立 TLS 连接。		
前提和约束	配置支持 X25519Kyber768Draft00 的 TLS 客户端和服务端程序。		

测试步骤

序号	输入及操作	期望结果与评估标准	实测结果
1.	启动支持 X25519Kyber768Draft00 的 TLS 服务端程序。	服务器接收 ClientHello 消息	
2.	客户端配置 X25519Kyber768Draft00 并发起 TLS 1.3 连接。	客户端和服务端完成 TLS 握手。	
3.	查看客户端输出日志或协商结果记录。	实际协商的密钥交换组为 X25519Kyber768Draft00。	

测试阶段	☐自测 ☐第三方测试		
通过情况		**测试人员**	
测试时间		**测试地点**	

1.2 协议握手消息格式测试验证

测试用例名称	ClientHello 消息格式验证	标识	TLS-1-4
测试用例描述	发送正确格式的 ClientHello 消息，包含多个支持的协议版本和加密交换组，”SM2Kyber512Draft00”、“X25519Kyber768Draft00”等字段，以及混合公钥，是否合规。预期服务器返回正确的 ServerHello 消息，选择合适的协议版本和密钥交换参数。		
前提和约束	配置支持 TLS 协议的服务器软件		

测试步骤			
序号	输入及操作	期望结果与评估标准	实测结果
1.	客户端发送正确格式的 ClientHello 消息，包含多个支持的协议版本和加密交换组以及混合公钥	服务器接收 ClientHello 消息	
2.	服务器接收 ClientHello 消息，选择合适的协议版本和密钥交换参数	服务器返回正确的 ServerHello 消息。	

测试阶段	□自测 □第三方测试		
通过情况		测试人员	
测试时间		测试地点	

测试用例名称	ServerHello 消息格式验证	标识	TLS-1-5
测试用例描述	检查服务器接收到 ClientHello 后，发送的 ServerHello 消息中是否对应协商参数：X25519 公钥与 Kyber 密文或 SM2 公钥与 Kyber 密文等。以及协议版本、选择的加密交换组等是否正确响应。		
前提和约束	配置支持 TLS 协议的服务器软件		

测试步骤			
序号	输入及操作	期望结果与评估标准	实测结果
1.	检查服务器接收到 ClientHello 后，发送的 ServerHello 消息中是否对应协商参数	检查通过	
2.	检查协议版本、选择的加密交换组等是否正确响应	检查通过	

测试阶段	□自测 □第三方测试		
通过情况		测试人员	

测试时间		测试地点	

1.3 密钥交换协商策略验证

测试用例名称	多密钥交换组协商策略验证	标识	TLS-1-6
测试用例描述	客户端和服务端同时配置多个混合密钥交换组，包括SM2Kyber512Draft00、X25519Kyber512Draft00、X25519Kyber768Draft00等，发起 TLS 1.3 握手连接，验证协议能够从双方共同支持的密钥交换组中完成协商，并记录实际协商结果。		
前提和约束	配置支持多个混合密钥交换组的 TLS 客户端和服务端程序。		

测试步骤

序号	输入及操作	期望结果与评估标准	实测结果
1.	启动配置多个混合密钥交换组的 TLS 服务端程序。	服务端启动成功，监听测试端口。	
2.	客户端配置多个混合密钥交换组并发起 TLS 1.3 连接。	客户端和服务端完成 TLS 握手。	
3.	查看客户端输出日志或协商结果记录。	实际协商结果为双方共同支持的密钥交换组。	

测试阶段	□自测 □第三方测试		
通过情况		测试人员	
测试时间		测试地点	

1.4 异常交互处理测试验证

测试用例名称	异常协议版本处理	标识	TLS-1-7
测试用例描述	发送 ClientHello 消息时，故意设置错误的协议版本格式，预期服务器返回错误响应或拒绝连接。		
前提和约束	配置支持 TLS 协议的服务器软件		

测试步骤

序号	输入及操作	期望结果与评估标准	实测结果
1.	发送故意设置错误的协议版本格式的 ClientHello 消息	服务器返回错误响应或拒绝连接	

测试阶段	☐自测　☐第三方测试		
通过情况		**测试人员**	
测试时间		**测试地点**	

测试用例名称	无效密钥交换组处理	**标识**	TLS-1-8
测试用例描述	发送 ClientHello 消息时，故意设置错误的密钥交换组，预期服务器返回 HelloretryRequest 请求新的密钥交换组。		
前提和约束	配置支持 TLS 协议的服务器软件		

测试步骤

序号	**输入及操作**	**期望结果与评估标准**	**实测结果**
1.	发送故意设置错误的协议版本格式的 ClientHello 消息	服务器返回 HelloretryRequest 请求新的密钥交换组。	

测试阶段	☐自测　☐第三方测试		
通过情况		**测试人员**	
测试时间		**测试地点**	

1.5　证书校验测试验证

测试用例名称	证书有效性验证	**标识**	TLS-1-9
测试用例描述	提供无效的服务器证书（如证书过期、证书签名错误），预期客户端能正确识别并拒绝该证书，握手失败。		
前提和约束	配置支持 TLS 协议的服务器软件		

测试步骤

序号	**输入及操作**	**期望结果与评估标准**	**实测结果**
1.	提供无效的服务器证书	客户端正确识别并拒绝该证书，握手失败	

测试阶段	☐自测　☐第三方测试		
通过情况		**测试人员**	
测试时间		**测试地点**	

1.6 加密通信测试验证

测试用例名称	加密通信验证	标识	TLS-1-10
测试用例描述	在握手完成后，发送不同类型的数据（文本、二进制等），验证数据在传输过程中是否被正确加密和解密。		
前提和约束	配置支持 TLS 协议的服务器软件		

测试步骤			
序号	输入及操作	期望结果与评估标准	实测结果
1.	握手完成后，发送不同类型的数据，验证数据在传输过程中是否被正确加密和解密	数据在传输过程中能被正确加密和解密	

测试阶段	☐自测 ☐第三方测试		
通过情况		测试人员	
测试时间		测试地点	

2 混合抗量子 TLS 协议性能测试

2.1 TLS 单连接性能测试

测试用例名称	SM2Kyber512Draft00 单连接建立耗时	标识	TLS-2-1
测试用例描述	客户端和服务端均配置使用 SM2Kyber512Draft00 混合密钥交换组，重复建立单个 TLS 1.3 连接，记录每次连接建立耗时，统计平均连接建立时间。		
前提和约束	配置支持 TLS 协议的服务器软件		

测试步骤			
序号	输入及操作	期望结果与评估标准	实测结果
1.	启动支持 SM2Kyber512Draft00 的 TLS 服务端程序。	服务端启动成功，监听测试端口。	
2.	客户端多次发起 TLS 1.3 单连接请求，并记录每次连接建立耗时。	每次连接能够成功建立，并输出连接建立耗时。	
3.	汇总多次测试结果，计算平均耗时。	形成单连接建立耗时统计结果。	

测试阶段	☐自测 ☐第三方测试		
通过情况		测试人员	

测试时间		测试地点	

测试用例名称	X25519Kyber512Draft00 单连接建立耗时	标识	TLS-2-2
测试用例描述	客户端和服务端均配置使用 X25519Kyber512Draft00 混合密钥交换组，重复建立单个 TLS 1.3 连接，记录每次连接建立耗时，统计平均连接建立时间。		
前提和约束	配置支持 X25519Kyber512Draft00 的 TLS 客户端和服务端程序。		

测试步骤

序号	输入及操作	期望结果与评估标准	实测结果
1.	启动支持 X25519Kyber512Draft00 的 TLS 服务端程序。	服务端启动成功，监听测试端口。	
2.	客户端多次发起 TLS 1.3 单连接请求，并记录每次连接建立耗时。	每次连接能够成功建立，并输出连接建立耗时。	
3.	汇总多次测试结果，计算平均耗时。	形成单连接建立耗时统计结果。	

测试阶段	□自测 □第三方测试		
通过情况		测试人员	
测试时间		测试地点	

测试用例名称	X25519Kyber768Draft00 单连接建立耗时	标识	TLS-2-3
测试用例描述	客户端和服务端均配置使用 X25519Kyber768Draft00 混合密钥交换组，重复建立单个 TLS 1.3 连接，记录每次连接建立耗时，统计平均连接建立时间。		
前提和约束	配置支持 X25519Kyber768Draft00 的 TLS 客户端和服务端程序。		

测试步骤

序号	输入及操作	期望结果与评估标准	实测结果
1.	启动支持 X25519Kyber768Draft00 的 TLS 服务端程序。	服务端启动成功，监听测试端口。	
2.	客户端多次发起 TLS 1.3 单连接请求，并记录每次连接建立耗时。	每次连接能够成功建立，并输出连接建立耗时。	
3.	汇总多次测试结果，计算平均耗时。	形成单连接建立耗时统计结果。	

测试阶段	□自测 □第三方测试

通过情况		**测试人员**	
测试时间		**测试地点**	

2.2 TLS 多连接性能测试

测试用例名称	并发多个 TLS 连接建立耗时	**标识**	TLS-2-4
测试用例描述	测试系统在高并发场景下的连接建立性能，评估连接建立的成功率和平均耗时。		
前提和约束	配置支持 TLS 协议的服务器软件		

测试步骤

序号	**输入及操作**	**期望结果与评估标准**	**实测结果**
1.	测试系统在高并发场景下的连接建立性能	接建立的成功率和平均耗时符合设计要求	

测试阶段	□自测 □第三方测试		
通过情况		**测试人员**	
测试时间		**测试地点**	

2.3 服务器资源消耗测试

测试用例名称	服务器资源消耗	**标识**	TLS-2-5
测试用例描述	在 TLS 通信过程中，实时监测服务器的 CPU 使用率、内存占用情况，记录在不同通信阶段的资源消耗数据。		
前提和约束	配置支持 TLS 协议的服务器软件		

测试步骤

序号	**输入及操作**	**期望结果与评估标准**	**实测结果**
1	通信过程中，监测服务器的 CPU 使用率、内存占用情况和不同通信阶段的资源消耗数据	CPU 使用率、内存占用情况和不同通信阶段的资源消耗数据符合设计要求	

测试阶段	□自测 □第三方测试		
通过情况		**测试人员**	
测试时间		**测试地点**	

3 混合抗量子 TLS 协议兼容性测试

3.1 协商结果事件记录验证测试

测试用例名称	协商结果事件记录验证	标识	TLS-3-1
测试用例描述	客户端通过 CFEventTLS13NegotiatedKEX 记录 TLS 1.3 握手过程中实际协商的密钥交换组，用于验证测试结果与客户端、服务端配置预期一致。		
前提和约束	客户端程序启用协商结果事件记录，服务端配置支持对应密钥交换组。		

测试步骤			
序号	输入及操作	期望结果与评估标准	实测结果
1	启动支持混合密钥交换组的 TLS 服务端程序。	服务端启动成功，监听测试端口。	
2	客户端发起 TLS 1.3 连接，并启用协商事件记录。	客户端和服务端完成 TLS 握手。	
3	查看客户端输出日志。	日志中能够记录实际协商的密钥交换组，且与配置预期一致。	

测试阶段	□自测 □第三方测试		
通过情况		测试人员	
测试时间		测试地点	

3.2 密钥材料导出验证测试

测试用例名称	TLS ExportKeyingMaterial 密钥材料导出验证	标识	TLS-3-2
测试用例描述	握手完成后，客户端和服务端使用相同 label 调用 TLS ExportKeyingMaterial 导出 32 字节密钥材料，并将其作为 AES-GCM 应用层加密密钥，验证密钥导出功能能够支撑后续加密通信。		
前提和约束	客户端和服务端能够成功完成 TLS 1.3 握手，且双方配置相同的密钥导出 label。		

测试步骤			
序号	输入及操作	期望结果与评估标准	实测结果

1	客户端和服务端完成 TLS 1.3 握手连接。	TLS 连接建立成功。	
2	双方使用相同 label 导出 32 字节密钥材料。	密钥材料导出成功，无错误提示。	
3	使用导出的密钥材料进行 AES-GCM 加解密通信。	加密请求和加密响应能够正确解密。	

测试阶段	□自测 □第三方测试		
通过情况		**测试人员**	
测试时间		**测试地点**	

3.3 密钥交换回退验证测试

测试用例名称	X25519 传统密钥交换回退验证	**标识**	TLS-3-3
测试用例描述	在混合密钥交换组不可用但客户端和服务端均支持 X25519 的情况下，发起 TLS 1.3 握手连接，验证协议能够回退协商传统 X25519 密钥交换组。该用例仅用于兼容性验证，不计入抗量子混合算法覆盖。		
前提和约束	客户端和服务端均配置支持 X25519，并调整测试配置使混合密钥交换组不可用或不作为共同支持项。		

测试步骤

序号	输入及操作	期望结果与评估标准	实测结果
1	启动仅保留或优先使用 X25519 的 TLS 服务端程序。	服务端启动成功，监听测试端口。	
2	客户端发起 TLS 1.3 连接。	客户端和服务端完成 TLS 握手。	
3	查看客户端协商结果记录。	实际协商的密钥交换组为 X25519，并标注为传统算法兼容性回退。	

测试阶段	□自测 □第三方测试		
通过情况		**测试人员**	
测试时间		**测试地点**	

3.4 跨平台编译与运行验证测试

测试用例名称	Linux 平台编译与运行验证	标识	TLS-3-4
测试用例描述	在 Linux 环境下编译 TLS 协议源码库，生成 Linux 目标平台服务端和客户端可执行文件，并验证服务端和客户端能够完成 TLS 连接建立及加密通信。		
前提和约束	Linux 测试环境已安装 Go 编译环境，源码库和证书文件准备完毕。		

测试步骤			
序号	输入及操作	期望结果与评估标准	实测结果
1	在 Linux 环境下执行服务端和客户端编译命令。	编译成功，生成 Linux 目标平台可执行文件。	
2	启动 Linux 服务端程序。	服务端启动成功，监听测试端口。	
3	运行 Linux 客户端程序连接服务端。	客户端和服务端建立 TLS 连接成功，加密通信正常。	

测试阶段	□自测 □第三方测试		
通过情况		测试人员	
测试时间		测试地点	

测试用例名称	Windows 目标平台交叉编译验证	标识	TLS-3-5
测试用例描述	在 Linux 环境下对 TLS 协议源码库执行 Windows 目标平台交叉编译，生成 Windows 服务端和客户端可执行文件，验证源码库具备 Windows 目标平台编译适配能力。		
前提和约束	Linux 测试环境已安装 Go 编译环境，TLS 协议源码库准备完毕。		

测试步骤			
序号	输入及操作	期望结果与评估标准	实测结果
1	在 Linux 环境下执行 Windows 服务端目标平台交叉编译命令。	编译成功，生成 windows_server.exe。	
2	在 Linux 环境下执行 Windows 客户端目标平台	编译成功，生成	

	交叉编译命令。	windows_client.exe。	
3	检查 Windows 目标平台编译产物。	产物文件存在，编译过程无错误。	

测试阶段	☐自测 ☐第三方测试		
通过情况		**测试人员**	
测试时间		**测试地点**	

4 测试结果总结

序号	测试用例	测试结果
1	TLS-1-1	□通过 □不通过
2	TLS-1-2	□通过 □不通过
3	TLS-1-3	□通过 □不通过
4	TLS-1-4	□通过 □不通过
5	TLS-1-5	□通过 □不通过
6	TLS-1-6	□通过 □不通过
7	TLS-1-7	□通过 □不通过
8	TLS-1-8	□通过 □不通过
9	TLS-1-9	□通过 □不通过
10	TLS-1-10	□通过 □不通过
11	TLS-2-1	□通过 □不通过
12	TLS-2-2	□通过 □不通过
13	TLS-2-3	□通过 □不通过
14	TLS-2-4	□通过 □不通过
15	TLS-2-5	□通过 □不通过
16	TLS-3-1	□通过 □不通过
17	TLS-3-2	□通过 □不通过
18	TLS-3-3	□通过 □不通过
19	TLS-3-4	□通过 □不通过
20	TLS-3-5	□通过 □不通过
测试结果统计		测试___项，通过___项